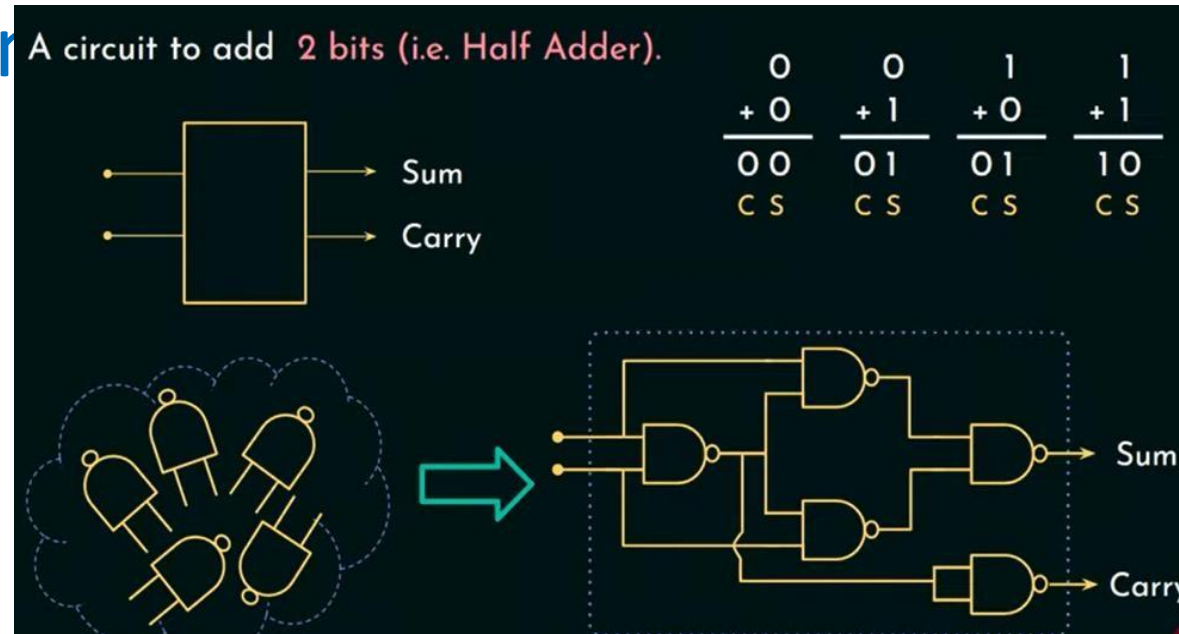


Computer Architecture and Organization



Computer Architecture	Computer Organization
Describes what the computer does (functional behaviour)	Describes how it does it (structural relationship)
For Designing a computer- its architecture is fixed first	For Designing a computer- organization decided after its architecture
We will decide - Instruction sets, number of bits used for data representation, registers, data types and addressing modes	Consists of physical units like circuit designs, peripherals, Control signals, interfaces, memory technology
e.g. Is there a multiply instruction?	-e.g. Is there a hardware multiply unit or is it done by repeated addition?

- **Computer:** It is a fast electronic calculating machine that accepts digitized input information, processes it according to the list of internally stored instructions & produces the resulting output information. The list of internally stored instructions is called as a Computer program & internal storage is called as Computer memory.
- **Functional units:** A computer consists of five functionally independent parts: Input, output, ALU, memory & control units

Functional Units

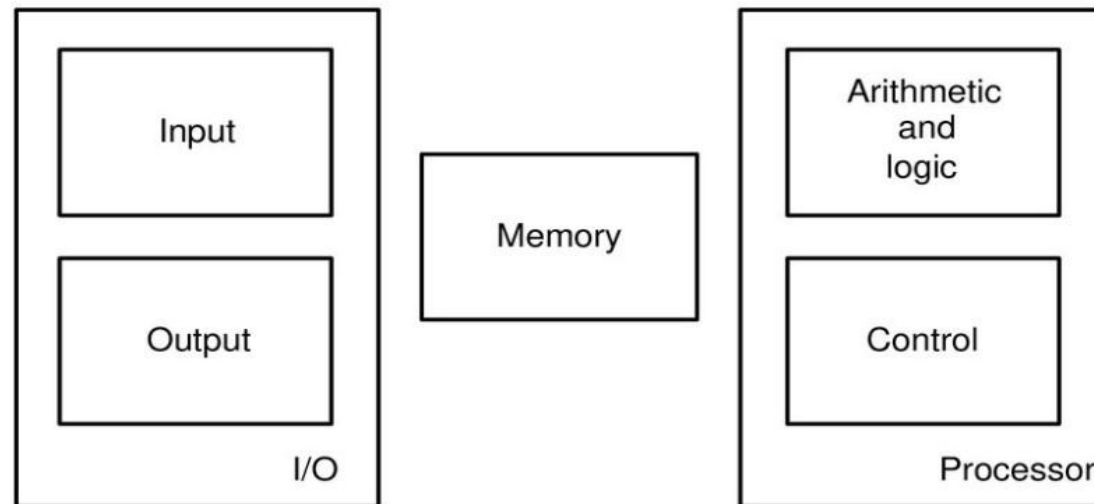


Figure 1.1. Basic functional units of a computer.

Input Unit

- Computers reads the data through input unit.
- The most common Input devices are Keyboard, joystick, trackballs, microphone and mouse
- When a key is pressed (keyboard), corresponding letter/digit is translated into its corresponding binary code and transmitted over a cable to either the memory or the processor.

Alphanumeric characters are also expressed in the form binary codes. Two commonly used codes are:

- **ASCII** (American Standard code for Information Interchange)
- **EBCDIC** (Extended Binary-Coded Decimal Interchange Code)

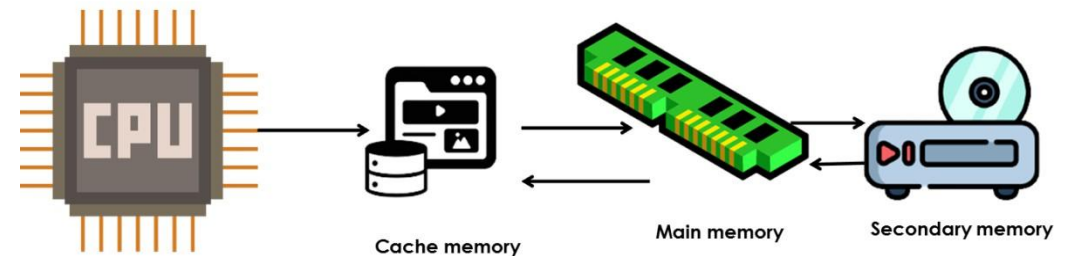
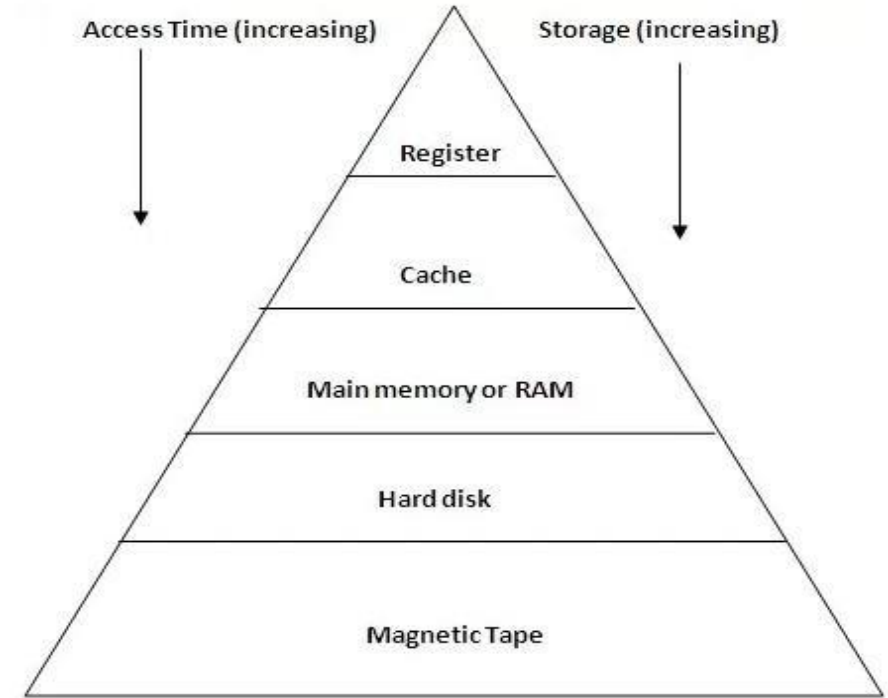
Memory Unit

- Function: is to store programs and data
- There are 2 classes of storage:
 - Primary/Main memory (RAM, cache memory)
 - Secondary (ROM, flash drives, hard disk drives, magnetic tapes)
- **Primary Storage**
 - Fast memory that operates at electronic speeds
 - The memory contains a large number of semiconductor storage cells, each capable of storing 1 bit of information
 - These cells are processed in groups of fixed size called **word**
 - The number of bits in each word is known as **word length**
 - Range from 16 to 64 bits
 - Word size describe the no. of bits of data processed by the microprocessor in one go (at a time) (Modern Computers: 32/64 bits).

Memory Unit

- Memory of a computer is normally implemented as a memory hierarchy of 3 or 4 levels of semiconductor RAM units with different speeds & sizes.
- Time required to access one word is called the **memory access time**
- The small, fast, RAM units are called caches.
- Register: used for temporary storage of calculations

Memory Hierarchy

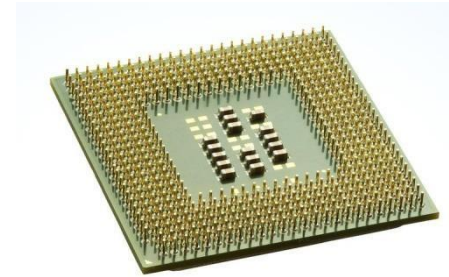


Memory Unit

- **Secondary Storage**

- Is used when large amount of data and many programs have to be stored
- It contains infrequently accessed information
- Additional & cheaper memory
- Ex: Magnetic disks and tapes & optical disks (CD-ROMs)

CPU: Arithmetic and Logic Unit



- ALU performs all the arithmetic and logic operations
- For ex: addition, multiplication, division, comparison etc
- Any operation is initiated by bringing the required operands into the processor, where the operation is performed by the ALU (Data transfer b/w memory & processor)
- Suppose two numbers located in the memory are to be added. They are brought into the processor, and the actual addition is carried out by the ALU. The sum may then be stored in the memory or retained in the processor for immediate use.
- When operands are brought into the processor, they are stored in high- speed storage elements called registers. Each register can store one word of data

Control Unit

- The memory, arithmetic and logic, and input and output units store and process information and perform input and output operations. **The operation of these units is coordinated by control unit.**
- The control unit is effectively the **nerve center** that sends control signals to other units and senses their states.
- It directs the operation of the other units by providing timing and control signals.
 - Timing signals are signals that determine when a given action is to take place.
 - Control signals supervise execution of the instructions

Output Unit

- Counterpart of Input unit
- Its function is to send processed results to outside world
- The familiar example of output device is printer (various types), monitor, speakers

Operation of a Computer - Summarized

- The computer accepts information in the form of pgms & data through an input unit & stores it in the memory
- Information stored in the memory is fetched, under pgm control, into an ALU, where it is processed
- Processed information leaves the computer through an Output unit
- All activities inside the machine are directed by the Control Unit

Basic Operational Concepts

❑ The information handled by a computer may be either *instruction* or *data*.

❑ **Instructions** are the explicit commands that

- Govern the transfer of information within a computer as well as between the computer and its I/O devices.
- Specify the arithmetic & logic operations to be performed.

Eg: Add LOCA, R0

Eg: ADD R1,R2

❑ A list of instructions that performs a task is called a **program**.

❑ Data are the numbers and encoded characters that are used as **operands** by the instructions.

Basic Operational Concepts

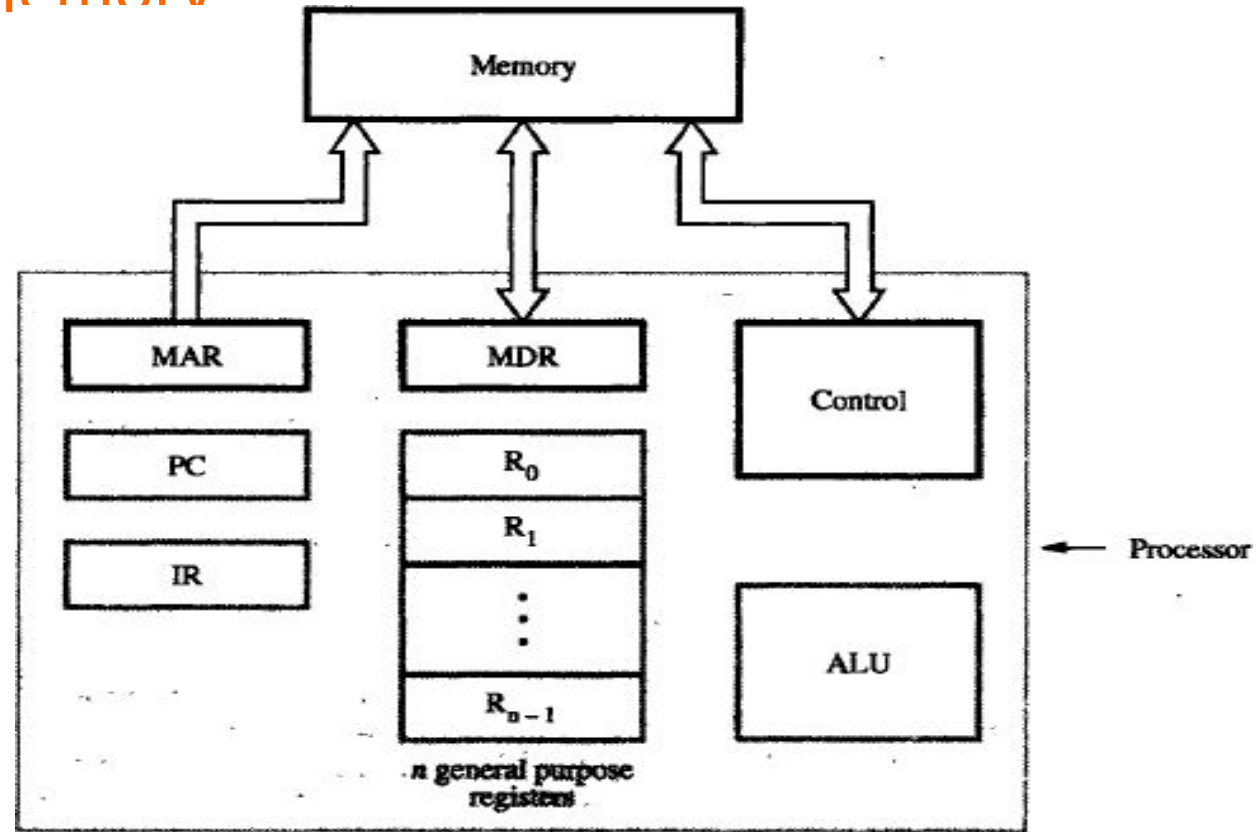
Consider the instruction **Add LOCA, R0**

This instruction adds the operand at memory location LOCA to the operand in a register in the processor, R0, and places the sum into register R0.

This instruction requires the performance of several steps.

- First, the instruction is fetched from the memory into the processor.
 - Next, the operand at LOCA is fetched and added to the contents of R0.
 - Finally, the resulting sum is stored in register R0.
-
- Transfers between the memory and the processor are started by sending the address of the memory location to be accessed to the memory unit and issuing the appropriate control signals. The data are then transferred to or from the memory.

Connection between the processor and memory



MAR - Memory Address Register

MDR - Memory Data Register

PC - Program Counter

IR - Instruction Register

Connection between the processor and memory

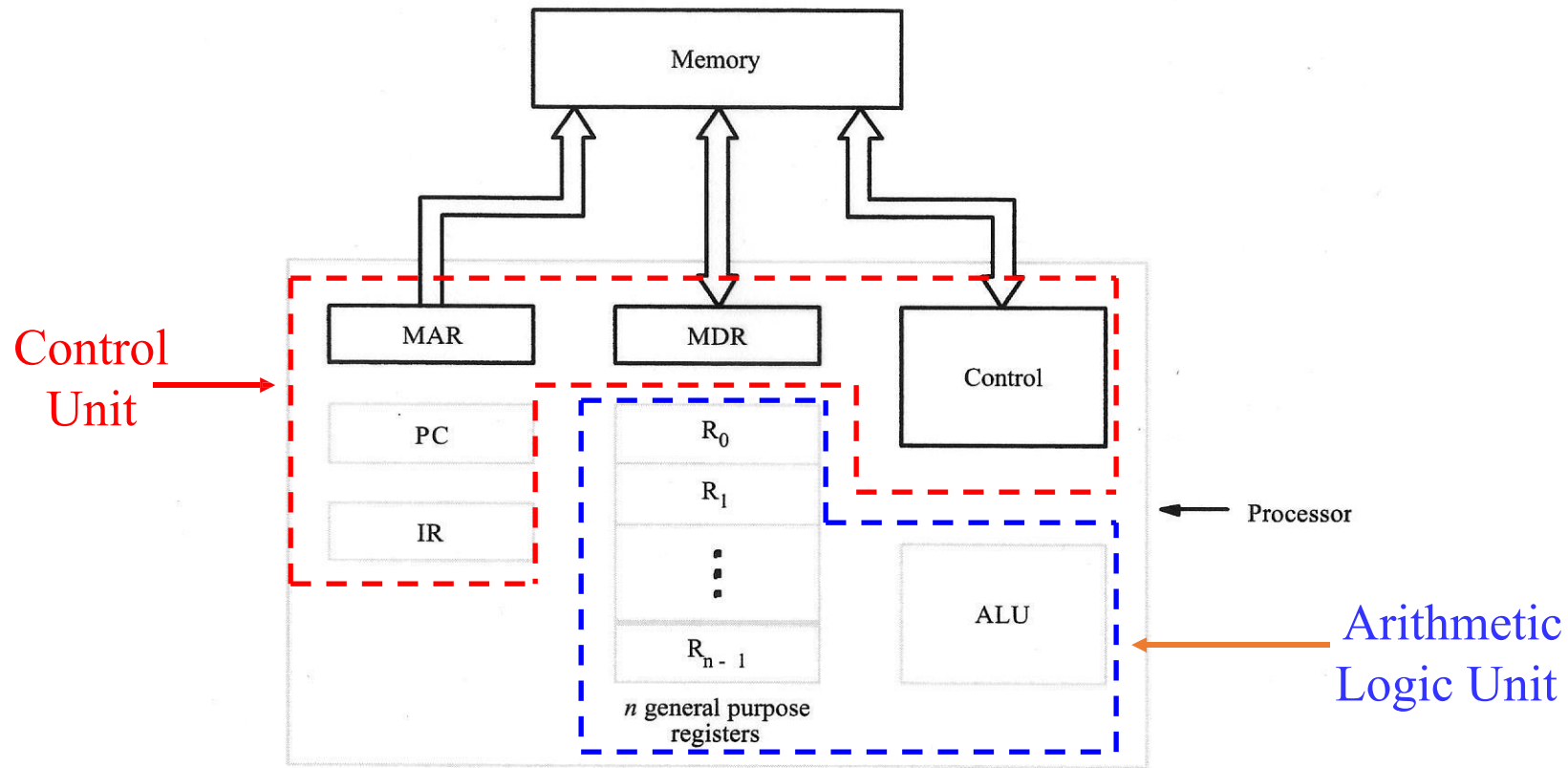


Figure 1.2. Connections between the processor and the memory.

MAR - Memory Address Register

MDR - Memory Data Register

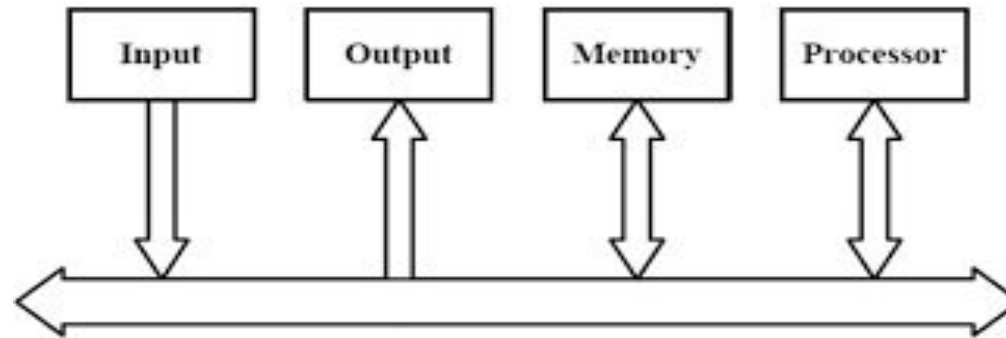
PC - Program Counter

IR - Instruction Register

Bus Structures

- **Bus:** Set of wires/lines, that interconnects all the components (subsystems) of a computer
- In addition to the lines that carry the data, the bus must have lines for address and control information.

Single Bus Structure



All units are connected to the same bus. Since the bus can be used for only one transfer at a time, only two units can actively use the bus at any given time.

Bus Structures

Single Bus Structure

Advantages:

- Simple
- Low cost
- Flexibility to attach peripheral devices.

Disadvantages:

- Concurrent operations cannot be performed.
- Lower performance

Multiple Bus Structure

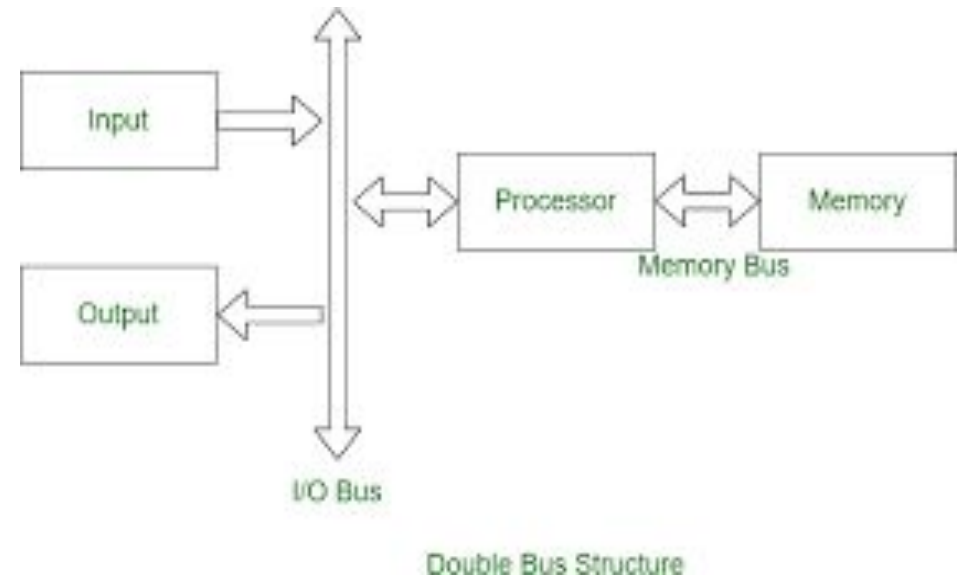
- Multiple Buses are used to interconnect functional units.

Advantages:

- Concurrent operations can be performed
- Higher performance

Disadvantages:

- Complex
- Costlier



Bus Structures

- The devices connected to a bus **vary widely in their speed of operation.**
 - Some electromechanical devices, such as keyboards and printers, are relatively slow.
 - Others, like magnetic or optical disks, are considerably faster.
- Results in
 - **Timing differences** among processors, memories, and I/O devices.
 - **High-speed processor being locked** to a slow I/O device during a sequence of data transfers.
- Solution- To smooth out the differences in timing among processors, memories, and external devices - **buffer registers are used**

Eg: Printing a file

- The processor sends the contents of the file over the bus to the printer buffer
- Since the buffer is an electronic register, this transfer requires relatively little time. Once the buffer is loaded, the printer can start printing without further intervention by the processor.
- The bus and the processor are no longer needed and can be used for other activities.

PERFORMANCE

The most important measure of the performance of a computer is how quickly it can execute programs.

The speed with which a computer executes programs is affected by:

1. The design of its hardware (memory and processor)
2. Its machine language instructions.
3. The compiler that translates programs (high level code) into machine language.

PERFORMANCE

. PROCESSOR CLOCK

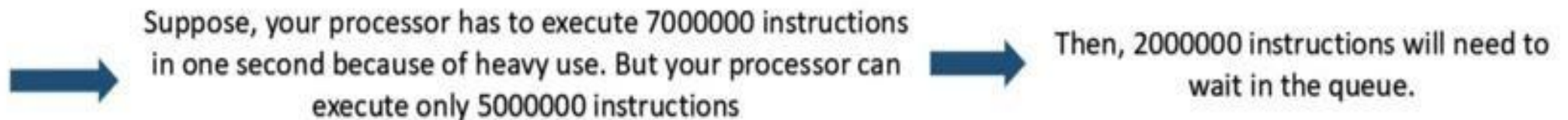
- Processor circuits are controlled by a timing signal called a *clock*.
- The clock defines regular time intervals, called *clock cycles*.
- To execute a machine instruction, the processor divides the action to be performed into a sequence of basic steps, such that each step can be completed in one clock cycle.
- The length P of one clock cycle is an important parameter that affects processor performance. Its inverse is the clock rate,

$R = 1/P$, which is measured in cycles per second.

System > About	
DESKTOP-VGDHLGU OptiPlex 5090	
Device specifications	
Device name	DESKTOP-VGDHLGU
Processor	11th Gen Intel(R) Core(TM) i7-11700 @ 2.50GHz 2.50 GHz
Installed RAM	16.0 GB (15.7 GB usable)
Device ID	2F0B4937-F55E-4A37-86F9-0EEAEC6FA560
Product ID	00331-90000-00001-AA316
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display

Note

Example



BASIC PERFORMANCE EQUATION

$$T = (N * S) / R$$

T-processor time required to execute a program

N-actual number of instruction executions

S-steps needed to execute one machine instruction
R-clock rate (R cycles per second)

To achieve **high performance**, the computer designer must seek ways to **reduce the value of T** , which means reducing N and S , and increasing R .

- The value of N is reduced if the source program is compiled into fewer machine instructions.
- The value of S is reduced if instructions have a smaller number of basic steps to perform or if the execution of instructions is overlapped.
- Using a higher-frequency clock increases the value of R , which means that the time required to complete a basic execution step is reduced.

Clock Rate

$$T = (N * S) / R$$

There are two possibilities for increasing the clock rate, R .

- First, improving the ***integrated-circuit (IC) technology*** makes **logic circuits** faster, which reduces the time needed to complete a basic step. This allows the clock period, P , to be reduced and the clock rate, R , to be increased.
- Second, **reducing the amount of processing done in one basic step** also makes it possible to reduce the clock period, P . However, if the actions that have to be performed by an instruction remain the same; the number of basic steps needed may increase.

PERFORMANCE MEASUREMENT

T (execution time) – measure the performance of a computer

- But computing the value of T is not simple. Parameters such as the clock speed (R) and various architectural features are not reliable indicators of the expected performance.
- Computer community adopted the idea of measuring computer performance using **benchmark programs**. The **performance measure** is the time it takes a computer to execute a given benchmark.
- A nonprofit organization called **System Performance Evaluation Corporation (SPEC)** selects the benchmark programs - game playing, compiler, and database applications to numerically intensive programs in astrophysics and quantum chemistry.
- The program is compiled for the computer under test, and the running time on a real computer is measured. The same program is also compiled and run on one computer selected as a reference.

PERFORMANCE MEASUREMENT

$$\text{SPEC rating} = \frac{\text{Running time on the reference computer}}{\text{Running time on the computer under test}}$$

- SPEC rating of 50 means that the computer under test is 50 times faster than the reference computer. The test is repeated for all the programs in the SPEC suite, and the geometric mean of the results is computed.

- The overall SPEC rating for the computer is given by
Where ***n*** is the number of programs in the suite.

$$\text{SPEC rating} = \left(\prod_{i=1}^n \text{SPEC}_i \right)^{\frac{1}{n}}$$

- SPEC rating is a measure of the combined effect of all the factors affecting performance, including the compiler, the operating system, the processor, and the memory of the computer being tested

$$\left(\prod_{i=1}^n x_i \right)^{\frac{1}{n}} = \sqrt[n]{x_1 x_2 \dots x_n}$$

Note

:

RISC (Reduced Instruction Set Computer) Architectures
CISC (Complex Instruction Set Computer) Architectures

Parameter	RISC	CISC
Instruction types	Simple	Complex
Number of instructions	Reduced (30-40)	Extended (100-200)
Duration of an instruction	One cycle	More cycles (4-120)
Instruction format	Fixed	Variable
Instruction execution	In parallel (pipeline)	Sequential
Addressing modes	Simple	Complex
Instructions accessing the memory	Two: Load and Store	Almost all from the set
Register set	multiple	unique
Complexity	In compiler	In CPU (micro-program)

RISC: Used in mobile phones , used in Apple
CISC: Laptops, used in Intel, AMD

CISC
mult A,B

load m1,A
load m2,B
Prod A,B
Store m3,A

Problem

S

1. For a RISC machine, the effective value of S is 1.25 and the average value of N is 200. If the clock rate is 500 MHz, calculate the total program execution time.

$$T = (N \times S)/R = (200 \times 1.25)/(500 \times 10^6) = 0.5 \mu s$$

2. The effective value of S for a RISC machine is 1.2 and for CISC is 1.5. Both machines have the same clock rate R. The time for execution on CISC machine is to be no longer than that of RISC machine. For this to happen, what is the largest allowable value for N, i.e., no. of instructions executed on CISC machine expressed as a percentage of N value for the RISC machine.

$$\begin{aligned} \text{Given, } S_c &= 1.5, S_R = 1.2, R_C = R_R = R \\ (1.2 \times N_R)/R &= (1.5 \times N_C)/R \\ 1.2N_R &= 1.5N_C \\ N_C/N_R &= 0.8 \end{aligned}$$

No. of instructions executed on CISC machine is 80% of the no. of instructions executed on RISC machine.

3. Assuming that the reference computer is Ultra SPARC10 workstation with 300 MHz Ultra SPARC processor. A company has to purchase 1000 new computers, hence ordered testing of new computers with SPEC 2000. Following observations were made:

<u>Programs</u>	<u>Runtime on reference computer</u>	<u>Runtime on new computer</u>
1	50 mins	5 mins
2	75 mins	4 mins
3	60 mins	6 mins
4	30 mins	3 mins

- The company system manager will place the order for purchasing new computers only if the overall SPEC rating is at least 12. After the said test, will system manager place order for purchase of new computer?

$$SPEC_1 = 50/5 = 10$$

$$SPEC_2 = 75/4 = 18.75$$

$$SPEC_3 = 60/6 = 10$$

$$SPEC_4 = 30/3 = 10$$

$$\text{Overall SPEC rating} = (\prod_1^n SPEC_i)^{1/n} = 11.70$$

Since overall SPEC rating is 11.70, the purchase order will not be placed.

4. List the steps needed to execute the machine instruction **Add LOCA,R0** in terms of transfers between the components of the processor and some control commands. Assume that the instruction itself is stored in the memory at location INSTR and that this address is initially in register PC. Also include the steps needed to update the contents of PC from INSTR to INSTR+1 so that the next instruction can be fetched.

- Transfer the contents of PC to MAR.
- Issue Read signal to memory and wait until requested word is transferred into MDR.
- Transfer MDR content to IR and decode it.
- Transfer the address LOCA to MAR.
- Issue Read signal and wait until MDR is loaded with the data.
- Transfer MDR content to the ALU.
- Transfer contents of R0 to the ALU.
- Perform addition of the two operands in the ALU and transfer the result into R0.
- Transfer the contents of PC to ALU.
- Add 4 to operand in ALU and transfer incremented address to PC.

MEMORY LOCATIONS AND ADDRESSES

- Memory consists of many millions of storage *cells*, each of which can store a bit of information having the value 0 or 1.
- Bits are not handled individually, a single bit represents a very small amount of information
- The usual approach is to deal with them in groups of fixed size.
- Each group of n bits is referred to as a *word* of information, and n is called the *word length*.

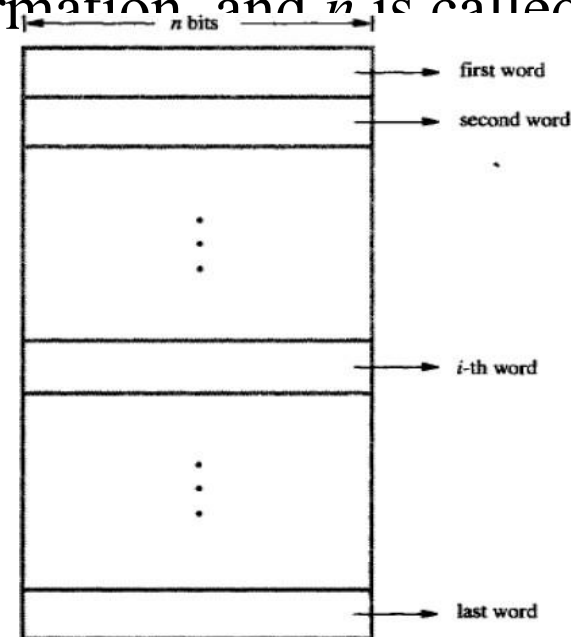


Figure 2.5 Memory words.

MEMORY LOCATIONS AND ADDRESSES

- The word length of the computer typically ranges from 16 to 64 bits.
- If the word length of a computer is 32 bits, a single word can store a 32-bit 2's-complement number or four ASCII characters, each occupying 8 bits. A unit of 8 bits is called a byte.

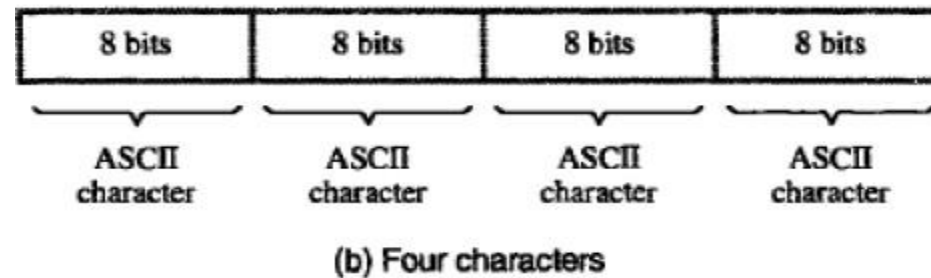
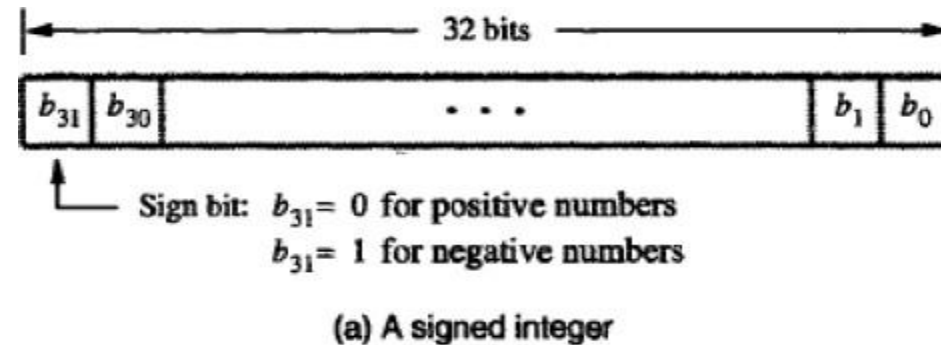


Figure 2.6 Examples of encoded information in a 32-bit word.

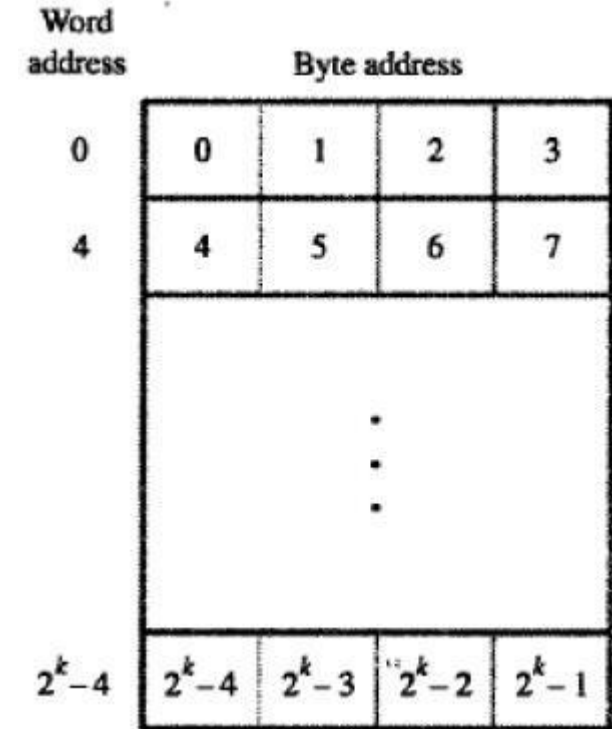
MEMORY LOCATIONS AND ADDRESSES

- Accessing the memory - Word or a byte, requires distinct names or *addresses* for each item location.
- Range is from 0 through 2^k-1 , for some suitable value of k .
- ***address space*** of the computer: 2^k addresses and the memory can have up to 2^k addressable locations.
- An address space is a range of valid addresses in memory that are available for a program or process
- 24-bit address generates an address space of 2^{24} (16,777,216) locations or 16M (16 Mega) , where 1M is the number 2^{20} (1,048,576).
- A 32-bit address creates an address space of 2^{32} (4294967296) or 4G (4 Giga) locations, where 1G is 2^{30} .
- Tera- 2^{40}

BYTE

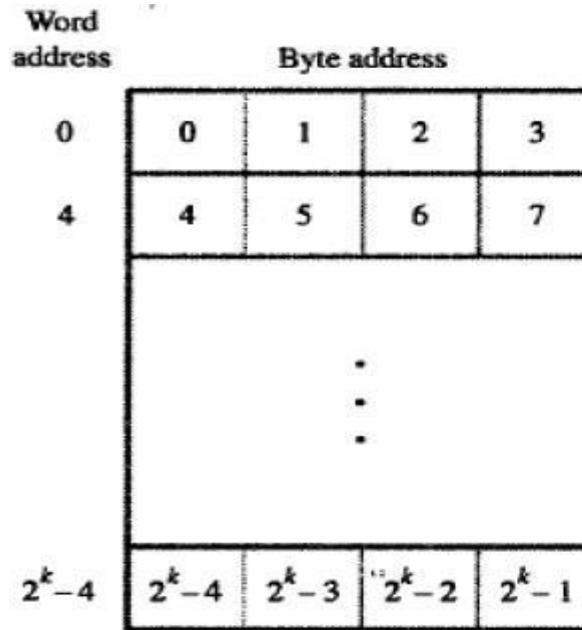
ADDRESSABILITY

- A byte is always 8 bits, but the word length typically ranges from 16 to 64 bits.
- It is impractical to assign distinct addresses to individual bit locations in the memory. The most practical assignment is to have successive addresses refer to **successive byte locations in the memory** - term byte-addressable memory is used for this assignment
- Byte locations have addresses 0, 1, 2, Thus, if the word length of the machine is 32 bits, successive words are located at addresses 0, 4, 8, . . . with each word consisting of four bytes.

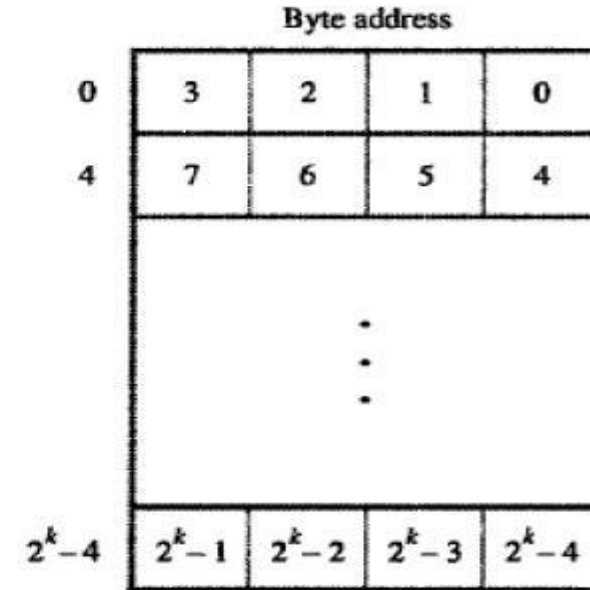


BIG-ENDIAN AND LITTLE-ENDIAN ASSIGNMENTS

- Two ways that byte addresses can be assigned across words
- The name **big-endian** is used when lower byte addresses are used for the more significant bytes (the leftmost bytes) of the word.
- The name **little-endian** is used for the opposite ordering, where the lower byte addresses are used for the less significant bytes (the rightmost bytes) of the word.



(a) Big-endian assignment



(b) Little-endian assignment

WORD

ALIGNMENT

- If the word length is 32-bit, then word boundaries occur at addresses 0, 4, 8, 12, 16 . .
- We say that the word locations have *aligned* addresses. In general, ***words are said to be aligned*** in memory if they begin at a byte address that is a multiple of the number of bytes in a word.
- In general, the number of bytes in a word is a power of 2. Hence, if the word length is 16 (2 bytes), aligned words begin at byte addresses 0, 2, 4, 6. . . , and for a word length of 64 (2^3 bytes), aligned words begin at byte addresses 0, 8, 16,
- If words begin at an arbitrary byte address then they are said to have ***unaligned addresses***.

MEMORY

OPERATIONS

- Both program instructions and data operands are stored in the memory.
- To execute an instruction by processor control circuits - transfer is required between memory to the processor.
- Two basic operations involving the memory are ***Load*** (or *Read* or *Fetch*) and ***Store*** (or *Write*).

Load operation

- Transfers a copy of the contents of a specific **memory location to the processor**. The memory contents remain unchanged.
- To start a Load operation, the processor sends the address of the desired location to the memory and requests that its contents be read. The memory reads the data stored at that address and sends them to the processor.

Store operation

- Transfers an item of information from **the processor to a specific memory location**, destroying the initial contents of that location.
- The processor sends the address of the desired location to the memory, together with the data to be written into that location.

INSTRUCTIONS AND INSTRUCTION SEQUENCING

The instruction set provides commands to the processor, to tell it what it needs to do.

A computer must have instructions capable of performing four types of operations: (Instruction Set Architecture)

- Data transfers between the memory and the processor registers
- Arithmetic and logic operations on data
- Program sequencing and control
- I/O transfers

Not

Name	Mnemonic
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	IN
Output	OUT
Push	PUSH
Pop	POP

Data transfer instructions

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate	NEG

Arithmetic and Logical

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

❑ **TABLE 10-7**
Typical Program Control Instructions

Name	Mnemonic
Branch	BR
Jump	JMP
Call procedure	CALL
Return from procedure	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TEST

Program Control

Computer Instructions

- To transfer the information from one location in the computer to another the following locations are involved: **memory locations, processor registers, or registers in the I/O subsystem.**

The location is identified by a symbolic name. For example,

- names for the addresses of memory locations may be LOC, PLACE, A, VAR2;
- processor register names may be R0, R5; and
- I/O register names may be DATAIN, OUTSTATUS, and so on.

REGISTER TRANSFER

NOTATION

- The contents of a location are denoted by placing square brackets around the name of the location. Thus, the expression

$R1 \leftarrow [LOC]$

contents of memory location LOC
are transferred into processor register R1.

$R3 \leftarrow [R1] + [R2]$

- Transfers among registers
- The right-hand side of an RTN expression always denotes a value, and the left-hand side is the name of a location where the value is to be placed, overwriting the old contents of that location.

ASSEMBLY LANGUAGE NOTATION

Move LOC,R1

transfer from memory location LOC to processor register R1

Add R1,R2,R3

two numbers contained in processor registers R1 and R2 and placing their sum in R3

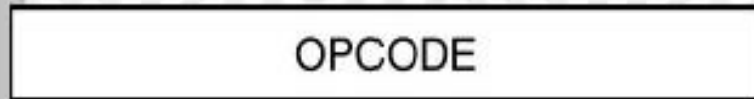
Assembly Language

- MOVE NUM1,R1
- MOVE #1,R2
- ADD #1,R1
- ADD R1,R2

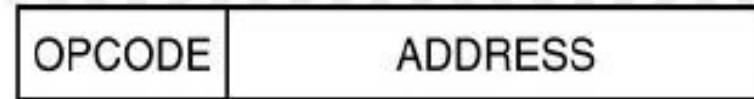
Register Transfer Notation

- $R1 \leftarrow [NUM1]$
- $R2 \leftarrow 1$
- $R1 \leftarrow 1 + [R1]$
- $R2 \leftarrow [R1] + [R2]$

BASIC INSTRUCTION TYPES (Common Instruction Formats)



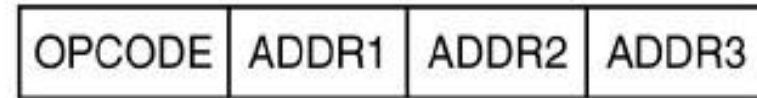
(a)



(b)



(c)



(d)

Four common instruction formats:

(a) Zero-address instruction.

(b) One-address instruction

(c) Two-address instruction.

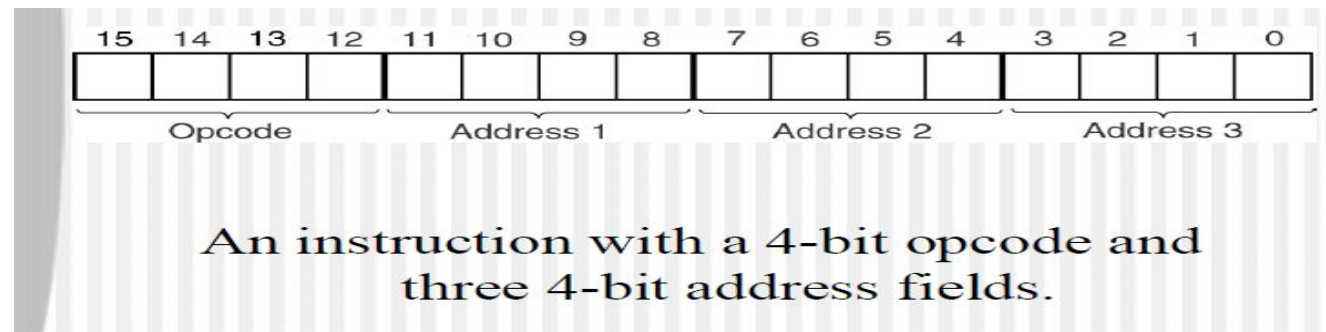
(d) Three-address instruction

Three-address

instruction:

Format: **Operation** **Source1,Source2,Destination**

- If k bits are needed to specify the memory address of each operand, the encoded form of the above instruction must contain $3k$ bits for addressing purposes in addition to the bits needed to denote the Add operation.



Example $X=(A+B)(C+D)$

ADD A, B,T1

ADD C, D, T2

MUL T1, T2,X

$M[A]+M[B] \rightarrow M[T1]$

$M[C]+M[D] \rightarrow M[T2]$

$M[T1]+M[T2] \rightarrow M[X]$

Two-address instruction:

- **Format:** **Operation Source, Destination**

Add A,B (B is both a source and a destination)

The problem in two address instruction is one of the operand is destroyed.

Move B,C

Add A,C

Example: $X = (A+B)(C+D)$

```
MOVE
A,T1 ADD
B,T1 MOVE
C,X ADD
D,X MUL
T1,X
```

```
M[A] → M[T1]
M[T1]+M[B] →
M[T1] M[C] → M[X]
M[X]+M[D] → M[X]
M[X]+M[T1] → M[X]
```

One-address

instruction:

- Machine instructions that specify only one memory operand. The second operand is implicitly specified in the instruction.
- A processor register, called the **accumulator**, is used for this purpose.
- An **accumulator** is primarily used as a register in a CPU to store intermediate logical or arithmetic data in multistep calculations.

Add A	adds the contents of memory location A to the contents of the accumulator register and place the sum back into the accumulator.
Load A	copies the contents of memory location A into the accumulator.
Store A	copies the contents of the accumulator into memory location A.

the operation $C \leftarrow [A] + [B]$ can be performed by

Load A

Add B

Store C

One-address instruction:

Example

$X = (A+B)(C+D)$

- ADD B
- ST X
- LD C
- ADD D
- MUL X
- ST X

$M[A] \rightarrow AC$

$ACC + M[B] \rightarrow AC$

$AC \rightarrow M[X]$

$M[C] \rightarrow AC$

$AC + M[D] \rightarrow AC$

$AC * M[X] \rightarrow AC$

$AC \rightarrow M[X]$

Zero-address instruction

- Stack architecture
- store operands in a structure called a pushdown stack

$C = A + B$

PUSH A
PUSH B
ADD
POP C



Zero-address instruction

Example

$X = (A+B)(C+D)$

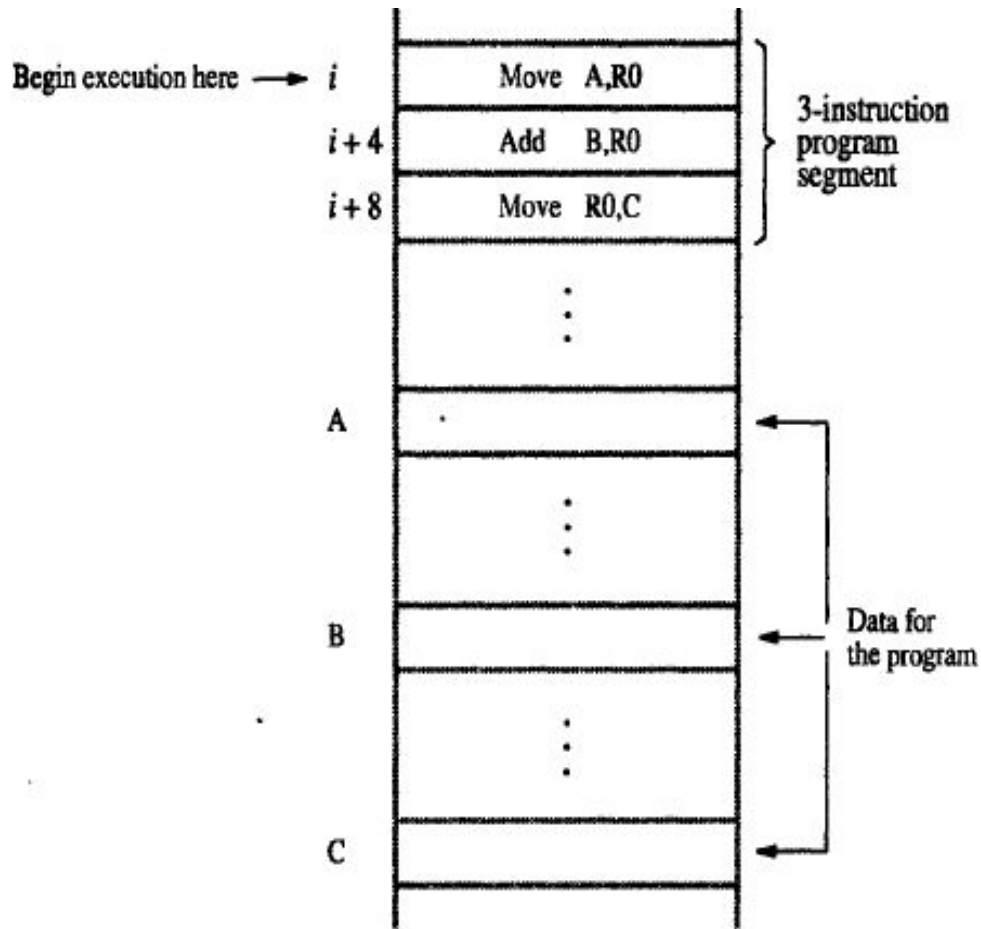
- PUSH B
- ADD
- PUSH C
- PUSH D
- ADD
- MUL
- POP X

$M[A] \rightarrow$
 $TOS\ M[B]$
 $\rightarrow TOS$
 $TOS+TOS-1 \rightarrow$
 $TOS\ M[C] \rightarrow TOS$
 $M[D] \rightarrow TOS$
 $TOS+TOS-1 \rightarrow$
 $TOS\ TOS * TOS-1$
 $\rightarrow TOS\ TOS \rightarrow$
 $M[X]$

TOS: Top of the Stack

INSTRUCTION EXECUTION AND STRAIGHT-LINE SEQUENCING

Consider the task, $C \leftarrow [A] + [B]$



- Word length is 32 bits and the memory is byte addressable
- The three instructions of the program are in successive word locations, starting at location i . Since each instruction is 4 bytes long, the second and third instructions start at addresses $i + 4$ and $i + 8$.

Figure 2.8 A program for $C \leftarrow [A] + [B]$.

INSTRUCTION EXECUTION AND STRAIGHT-LINE SEQUENCING

Consider the task, $C \leftarrow [A] + [B]$

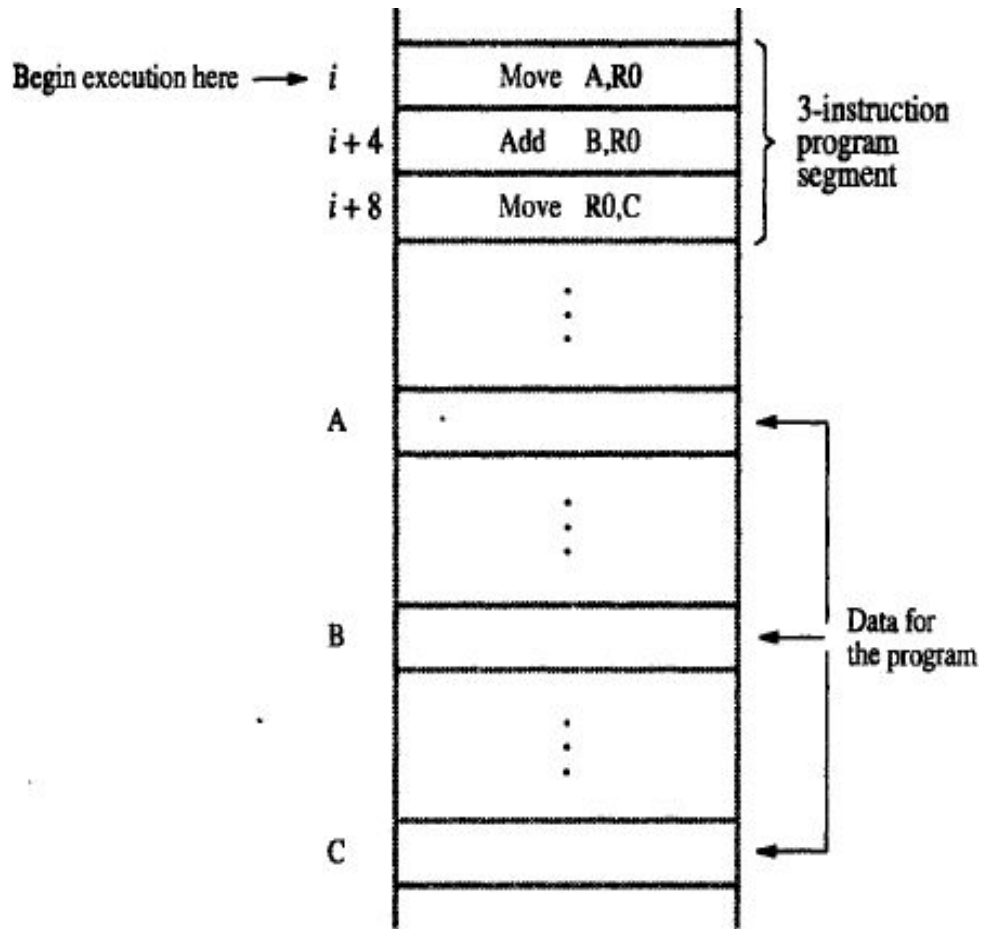
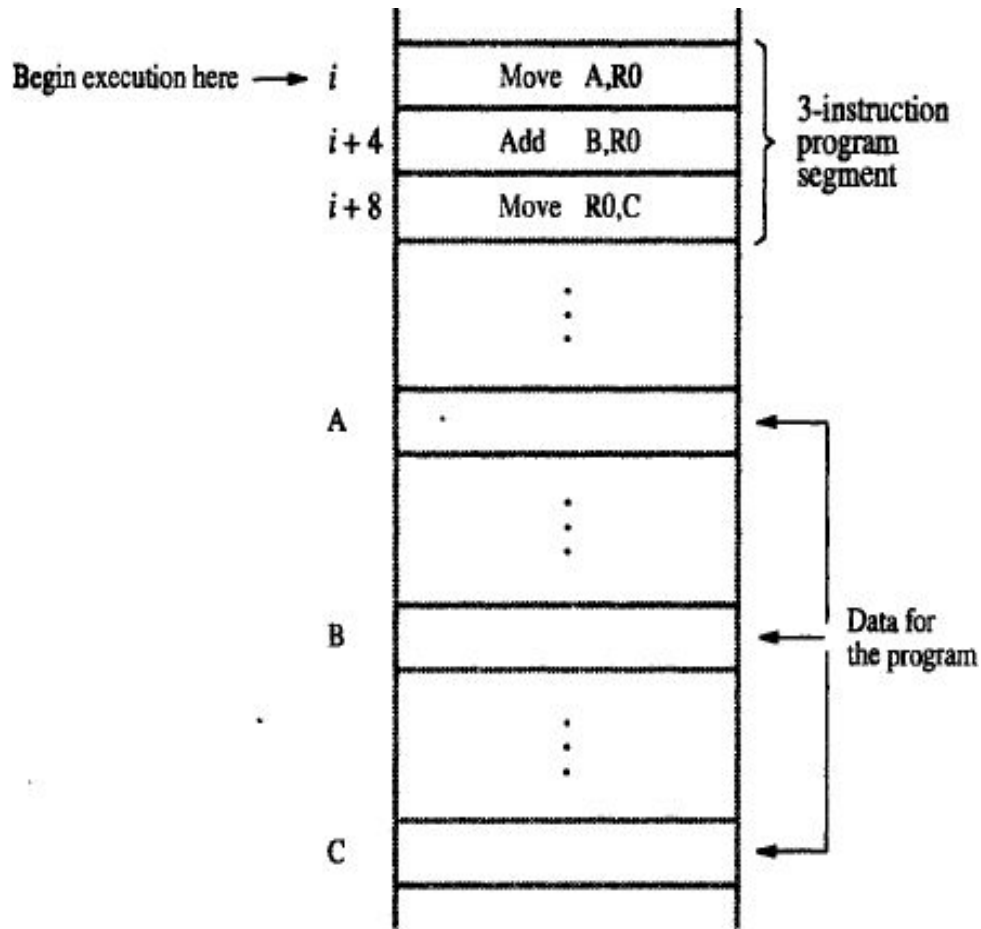


Figure 2.8 A program for $C \leftarrow [A] + [B]$.

- To begin executing a program, the address of its first instruction (i) must be placed into the PC.
- Then, processor control circuits use the information in the PC to fetch and execute instructions, **one at a time, in the order of increasing addresses. This is called Straight line sequencing**
- During the execution of each instruction, the PC is incremented by 4 to point to the next instruction.
- **Straight line sequencing:** means the instruction of a program is executed in a **sequential manner**(i.e. every time PC is incremented by a fixed offset). And no branch address is loaded on the PC

INSTRUCTION EXECUTION AND STRAIGHT-LINE SEQUENCING

Consider the task, $C \leftarrow [A] + [B]$



Executing a given instruction is a ***two-phase procedure***

Phase 1: instruction fetch

Registers : PC – MAR- Memory (read signal) – MDR- IR

Phase 2: instruction execute

- Determine which operation is to be performed (decode)
- Operation is then performed by the processor
- This often involves fetching operands from the memory or from processor registers, performing an arithmetic or logic operation, and storing the result in the destination location.
- PC - incremented

Figure 2.8 A program for $C \leftarrow [A] + [B]$.

BRANCHING

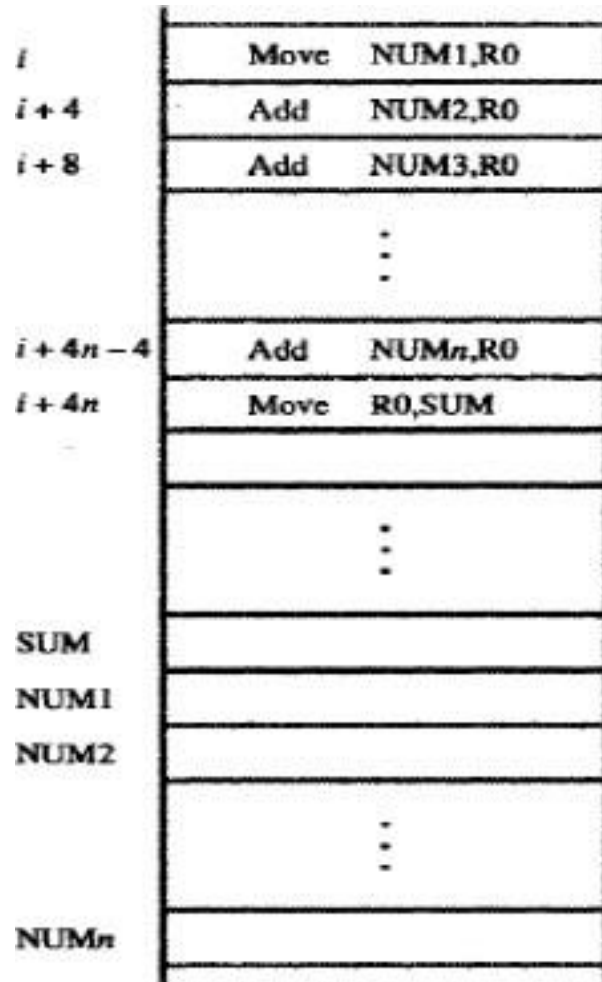


Figure 2.9 A straight-line program for adding n numbers.

Adding a list of n numbers

Addresses of the memory locations containing the n numbers are symbolically given as NUM1, NUM2, .. NUM n ,

Add instruction is used to add each number to the contents of register R0. The result is placed in memory location SUM

BRANCHIN

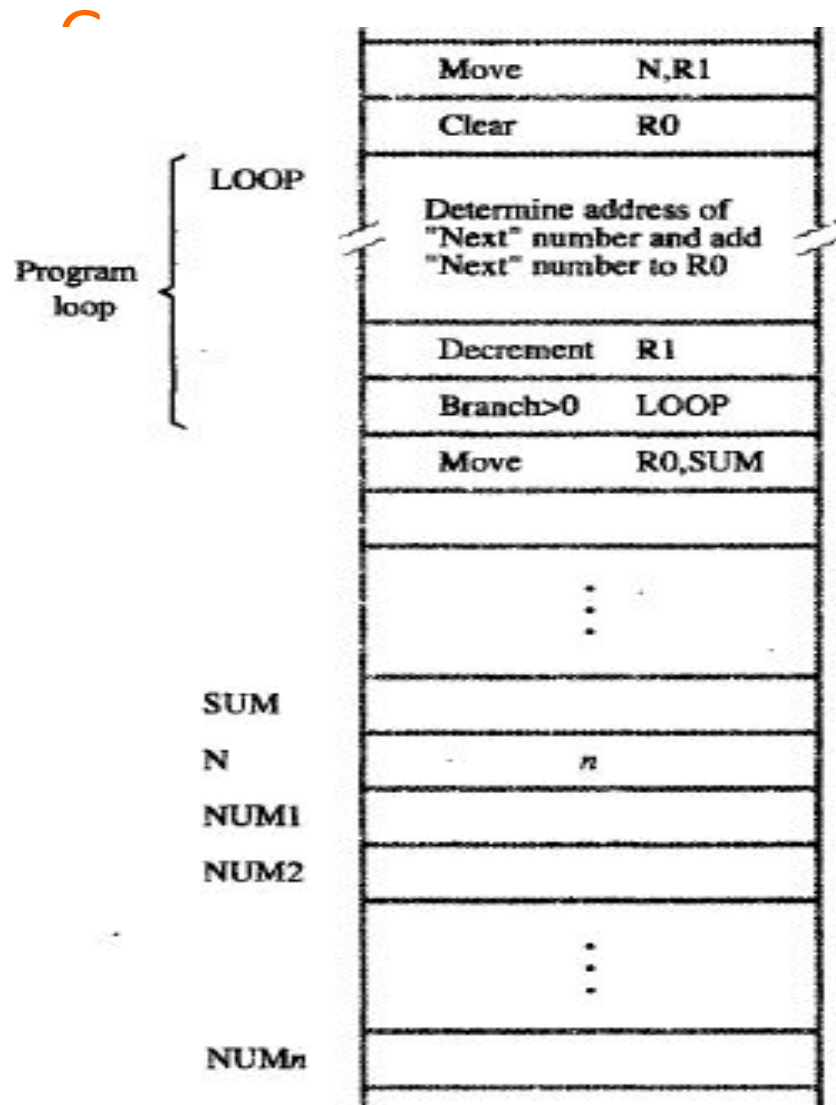


Figure 2.10 Using a loop to add n numbers.

- It is possible to place a **single Add instruction** in a program loop
- It starts at location LOOP and ends at the instruction Branch>0.
- During each pass through this loop, the address of the next list entry is determined, and that entry is fetched and added to R0.
 - Memory location **N - number of entries** in the list, n ,
 - Register **R1 is used as a counter** to determine the number of times the loop is executed.
 - Hence, the contents of location N are loaded into register R1 at the beginning of the program.
 - Decrement R1 reduces the contents of R1 by 1 each time through the loop.
 - Execution of the loop is repeated **as long as the result of the decrement operation is greater than zero.**

Branch instruction

- This type of instruction loads a new value into the program counter.
- As a result, the processor fetches and executes the instruction at this new address, called the ***branch target***, instead of the instruction at the location that follows the branch instruction in sequential address order.

Typical Program Control Instructions

Name	Mnemonic
Branch	BR
Jump	JMP
Call procedure	CALL
Return from procedure	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TEST

Conditional branch instruction

- Causes a branch only if a **specified condition is satisfied**.
- If the condition is not satisfied, the PC is incremented in the normal way, and the next instruction in sequential address order is fetched and executed.

Branch>0 **LOOP**

causes a branch to **location LOOP** if the result of the immediately preceding instruction, which is the decremented value in register R1, is greater than zero.

Branch Condition	Mnemonic	Test Condition
Branch if zero	BZ	$Z = 1$
Branch if not zero	BNZ	$Z = 0$
Branch if carry	BC	$C = 1$
Branch if no carry	BNC	$C = 0$
Branch if minus	BN	$N = 1$
Branch if plus	BNN	$N = 0$
Branch if overflow	BV	$V = 1$
Branch if no overflow	BNV	$V = 0$

CONDITION CODES

- Condition code flags in a processor register – **status register or condition code register**

Condition code register or status register

- Keeps track of information about the results of various operations for use by subsequent conditional branch instructions
- Condition code flags are set to 1 or cleared to 0, depending on the outcome of the operation performed.

Four commonly used flags are

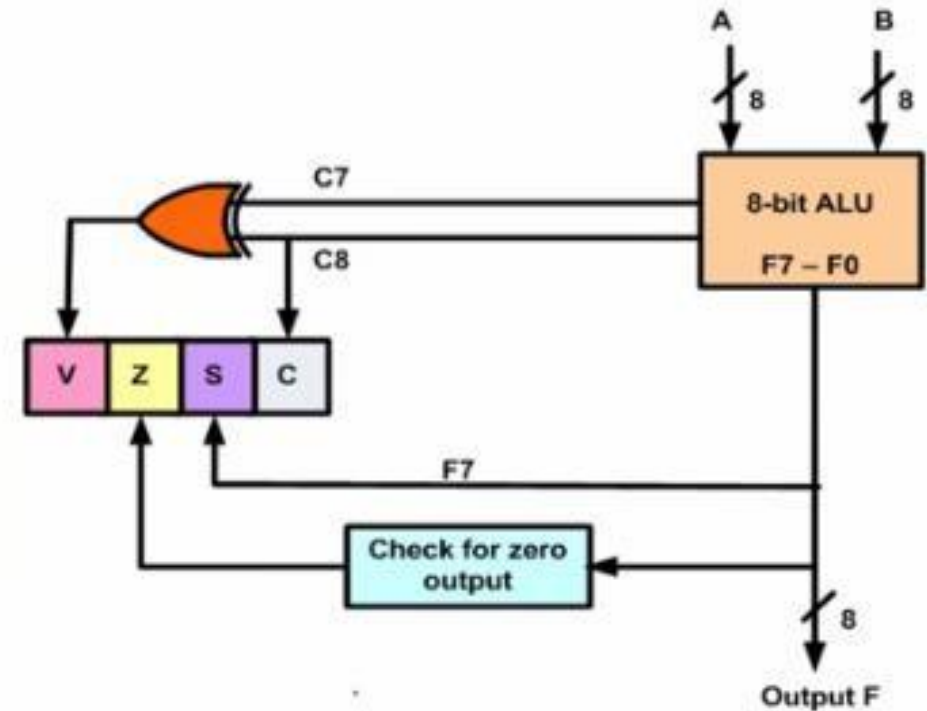
N (negative)	Set to 1 if the result of an arithmetic operation is negative; otherwise, cleared to 0.
Z (zero)	Z (zero) Set to 1 if the result of an arithmetic or logic operation is 0; otherwise, cleared to 0.
V (overflow)	Set to 1 if arithmetic overflow occurs i.e., if the result of an arithmetic operation is outside the representable range (-2^{n-1} to $+2^{n-1} - 1$ where n is the no. of bits used to represent the operands); otherwise, cleared to 0
C (carry)	Set to 1 if a carry occurs from the most significant bit position during an arithmetic operation.

Note

CONDITION CODES

status register

Status bit	Condition
Bit C (carry)	Set to 1 if the end carry C_8 is 1 Cleared to 0 if the end carry is 0
Bit S (sign)	Set to 1 if the highest-order bit F_7 is 1 Cleared to 0 if the bit is 0
Bit Z (zero)	Set to 1 if the output of the ALU contains all 0's Cleared to 0 otherwise
Bit V (overflow)	Set to 1 if the exclusive-OR of the last two carries is 1 Cleared to 0 otherwise



Addressing Modes

The different ways in which the location of an operand is specified in an instruction are referred to as *addressing modes*.

There are many different ways we can specify addresses. These are called addressing modes

- Direct Addressing mode
- In-Direct addressing mode
- Immediate Addressing mode
- Register Addressing mode
- Index addressing mode
- Relative addressing mode
- Auto Increment addressing mode
- Auto Decrement addressing mode

Addressing Modes

Mode	Assembly Convention	Register Transfer
Direct address	LD ADR	$AC \leftarrow M[ADR]$
Indirect address	LD @ADR	$AC \leftarrow M[M[ADR]]$
Relative address	LD \$ADR	$AC \leftarrow M[PC + ADR]$
Immediate operand	LD #NBR	$AC \leftarrow NBR$
Index addressing	LD ADR(X)	$AC \leftarrow M[ADR + XR]$
Register	LD R1	$AC \leftarrow R1$
Register indirect	LD (R1)	$AC \leftarrow M[R1]$
Autoincrement	LD (R1)+	$AC \leftarrow M[R1], R1 \leftarrow R1 + 1$

Direct & Indirect Addressing

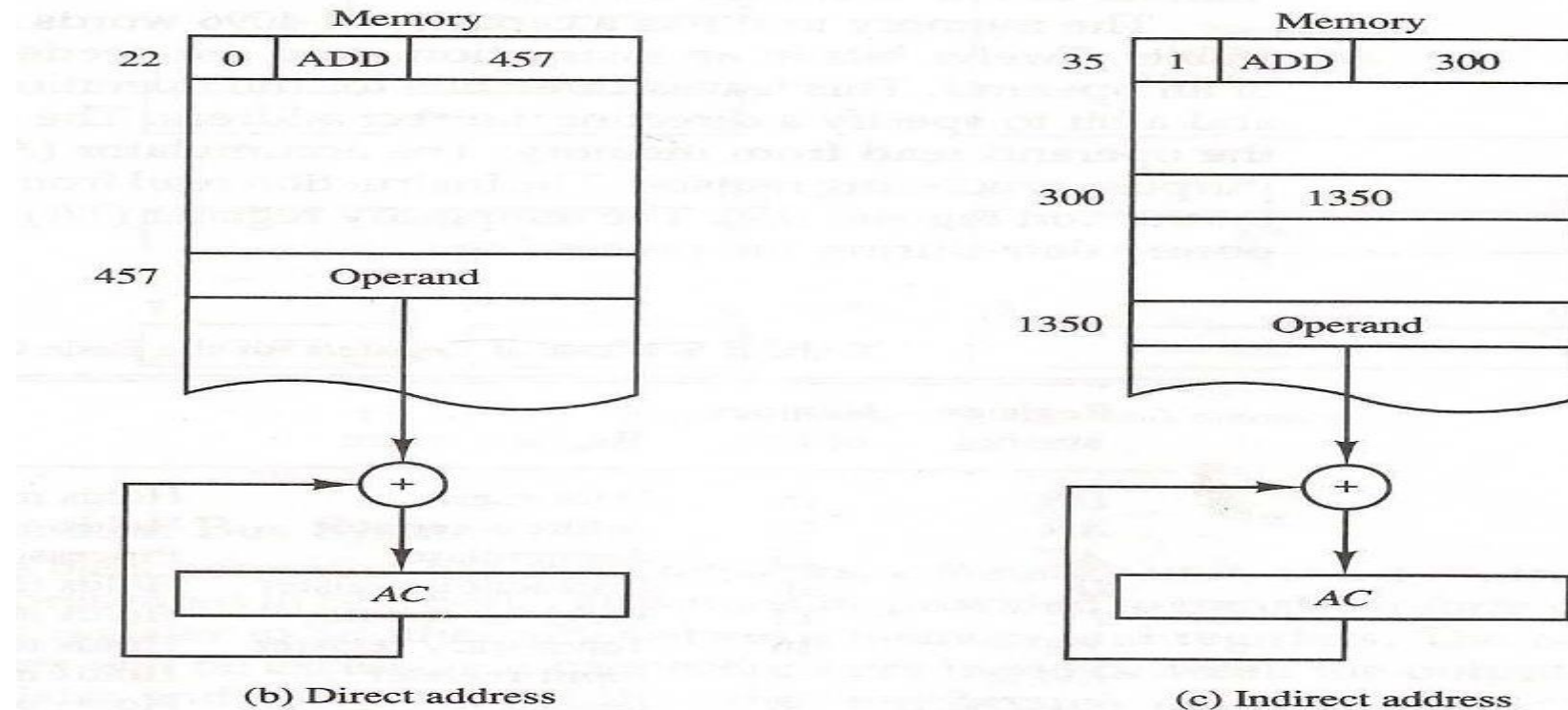


Figure 5-2 Demonstration of direct and indirect address.

In some addressing modes, the instruction does not give the operand or its address explicitly. Instead, it provides information from which the memory address of the operand can be determined. This address is the *effective address (EA)* of the operand.

Addressing Modes – Numerical Example

Address	Memory
200	Load to AC Mode
201	Address = 500
202	Next instruction
399	450
400	700
500	800
600	900
702	325
800	300

PC = 200

R1 = 400

XR = 100

AC

TABLE 8-4 Tabular List of Numerical Example

Addressing Mode	Effective Address	Content of AC
Direct address	500	800
Immediate operand	201	500
Indirect address	800	300
Relative address	702	325
Indexed address	600	900
Register	—	400
Register indirect	400	700
Autoincrement	400	700
Autodecrement	399	450

Figure 8-7 Numerical example for addressing modes.

IMPLEMENTATION OF VARIABLES AND CONSTANTS

- Variables and constants are the simplest data types found in every computer program.
- In assembly language, a variable is represented by allocating a **register** or a **memory location** to hold its value.

Ex : MOVE A, Ri

- **Register mode** — The operand is the contents of a processor register; the name (address) of the register is given in the instruction.

Ex: Load R1

- **Absolute mode (Direct addressing mode)** — The operand is in a memory location; the address of this location is given explicitly in the instruction.

Ex: Load A ,where A is the memory location

IMPLEMENTATION OF VARIABLES AND CONSTANTS

- Consider the representation of constants. Address and data constants can be represented in assembly language using the Immediate mode.
- **Immediate mode** — The operand is given explicitly in the instruction. EX:
Move #200, R0 (places the value 200 in register R0)

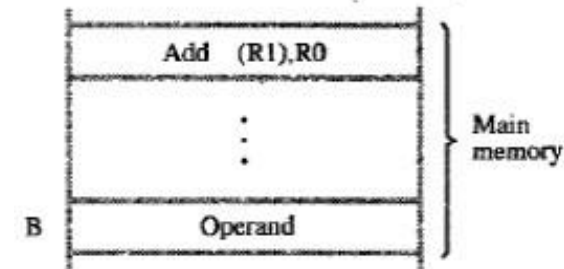
$A = B + 6$

- Move B,R1
- Add #6,R1
- Move R1,A

INDIRECTION AND POINTERS

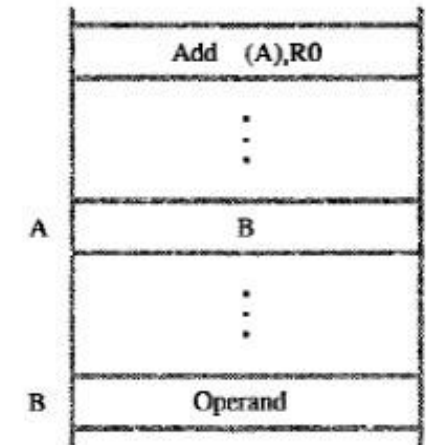
effective address (EA) : The instruction does not give the operand or its address explicitly. Instead, it provides information from which the memory address of the operand can be determined

- **Indirect mode** — The effective address of the operand is the contents of a register or memory location whose address appears in the instruction.
- The **indirection** is denoted by placing the name of the register or the memory address given in the instruction **in parentheses**.



R1 B Register

(a) Through a general-purpose register



(b) Through a memory location

Figure 2.11 Indirect addressing.

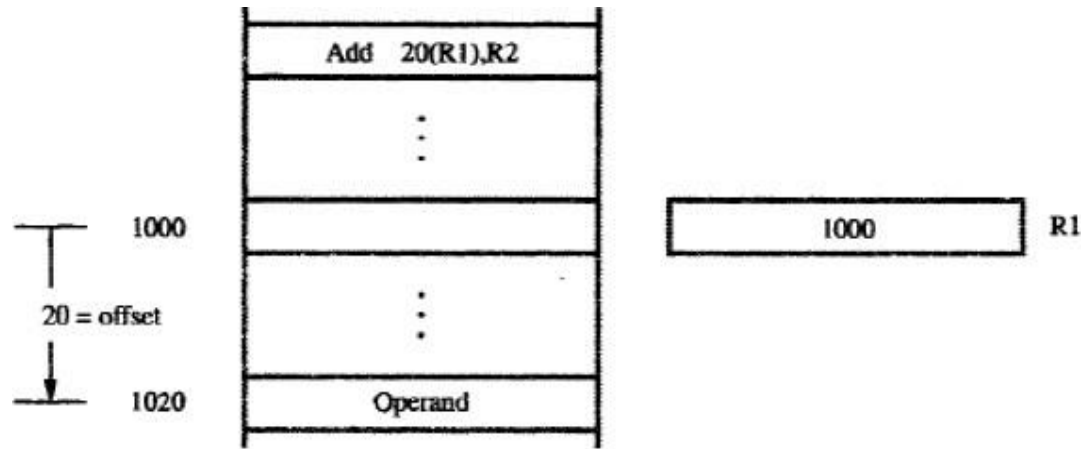
- (a) effective address of the operand: B
(b) effective address of the operand: B

INDEXING AND ARRAYS

- **Index mode** — The effective address of the operand is generated by adding a constant value to the contents of a register.
- *index register*- a special register provided for this purpose, or, more commonly, it may be any one of a set of general-purpose registers in the processor
- The Index mode is symbolically written as $X(R_i)$
- $EA = X + [R_i]$

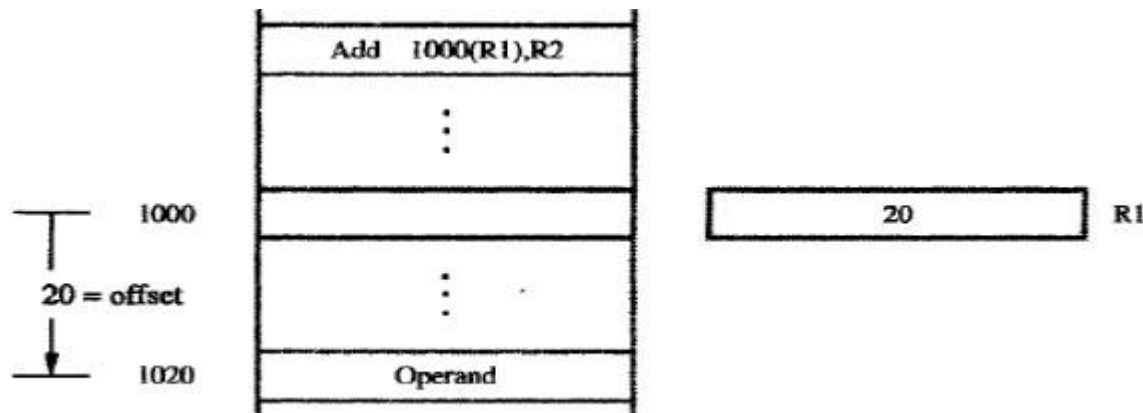
where, X denotes the constant value contained in the instruction and R_i is the name of the register involved.

INDEXING AND ARRAYS



(a) Offset is given as a constant

The index register, R1, contains the address of a memory location, and the value X defines an offset (or displacement) from this address to the location where the operand is present.



(b) Offset is in the index register

The constant X corresponds to a memory address, and the contents of the index register define the offset to the operand.

RELATIVE ADDRESSING

- **Relative mode** — The effective address is determined by the Index mode using the program counter in place of the general-purpose register Ri.
- The Relative mode is symbolically written as $X(PC)$
- $EA = [PC] + X$

Then, $X(PC)$ can be used to address a memory location that is X bytes away from the location presently pointed to by the program counter.

ADDITIONAL MODES

Useful for accessing data items in successive locations in the memory.

- **Autoincrement mode:** The effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next item in a list.
 - Autoincrement mode is written as $(Ri) +$
- **Auto decrement mode :** The contents of a register specified in the instruction are first automatically decremented and are then used as the effective address of the operand.
 - Autodecrement mode is written as $-(Ri)$

ADDITIONAL MODES

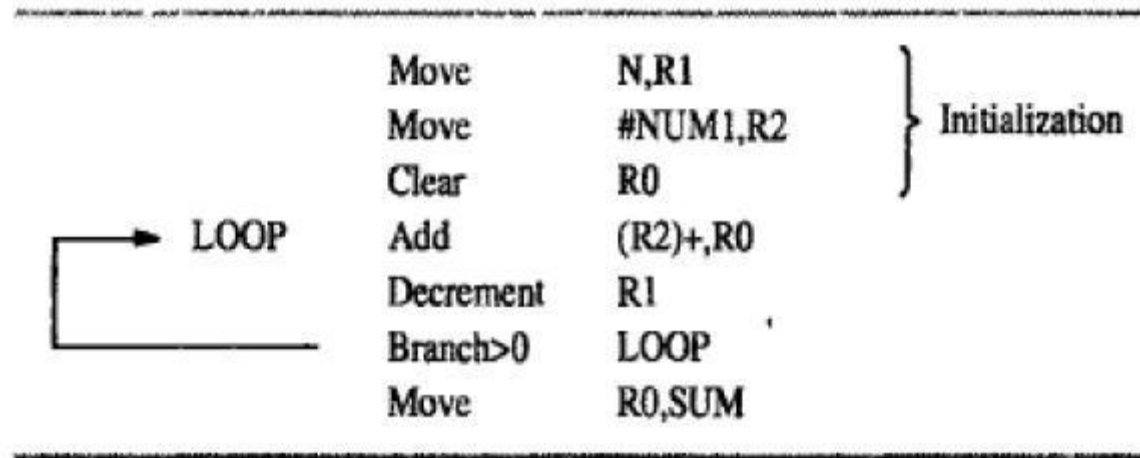


Figure 2.16 The Autoincrement addressing mode used in the program of Figure 2.12.



Generic Addressing Modes

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand=Value
Register	Ri	EA=Ri
Absolute (Direct)	LOC	EA=LOC
Indirect	(Ri)	EA=[Ri]
	(LOC)	EA=[LOC]
Index	X(Ri)	EA=[Ri]+X
Base with index	(Ri, Rj)	EA=[Ri]+[Rj]
Base with index and offset	X(Ri, Rj)	EA=[Ri]+[Rj]+X
Relative	X(PC)	EA=[PC]+X
Autoincrement	(Ri)+	EA=[Ri]; Increment Ri
Autodecrement	-(Ri)	Decrement Ri; EA=[Ri]

1)

A Program Counter contains a number 825 and address part of the instruction contains the number 24. The effective address in the relative addressing mode, when an instruction is read from the memory is

(A) 849

(B) 850

(C) 801

(D) 802

2)

Column-2:

1. Loops
2. Pointers
3. Constants

Column-1:

- X: Indirect addressing
Y: Immediate addressing
Z: Auto decrement addressing

- A. X-3, Y-2, Z-1
- B. X-1, Y-3, Z-2
- C. X-2, Y-3, Z-1
- D. X-3, Y-1, Z-2

3)

Sample GATE Question

Match each of the high level language statements given on the left hand side with the most natural addressing mode from those listed on the right hand side.

- | | |
|--------------------------------|------------------------|
| 1. <code>A[1] = B[J];</code> | a. Indirect addressing |
| 2. <code>while [*A++];</code> | b. Indexed addressing |
| 3. <code>int temp = *x;</code> | c. Autoincrement |

(A) (1, c), (2, b), (3, a)

(B) (1, a), (2, c), (3, b)

(C) (1, b), (2, c), (3, a)

(D) (1, a), (2, b), (3, c)

BASIC INPUT/OUTPUT OPERATIONS

Note

BASIC INPUT/OUTPUT OPERATIONS

Why Input/output Interface is required for transferring information between processor and peripherals?

Differences

Peripherals – electro mechanical devices
CPU or memory –electronic devices



Therefore, a conversion of signal values may be required

Data transfer rate

Peripherals - Usually slower

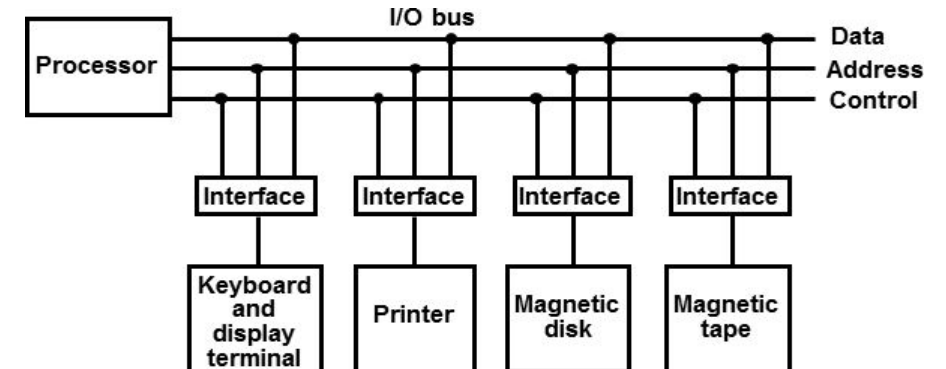
CPU or Memory - Usually faster than peripherals



Synchronization mechanism is needed

To resolve these differences, computer systems include special hardware components between the CPU and peripherals to supervise and synchronize all input and output transfers. These components are called interface units.

Interface: Point of contact between two parts of a system



BASIC INPUT/OUTPUT OPERATIONS



- Consider a task that reads in character input from a keyboard and produces character output on a display screen.
- One method to perform such I/O tasks is known as *program-controlled I/O*. Programmed- driven I/O means the program is polling or checking some hardware item.
- A program is written for the computer to check the flag in the status register to determine if a byte has been placed in the data register by the IO device. Flag =1, the CPU reads the data from the data register.

Three different Data Transfer Modes between the central computer (CPU or Memory) and peripherals (I/O devices):

- Programmed I/O
- Interrupt-Initiated IO
- Direct memory access (DMA)

Programmed I/O

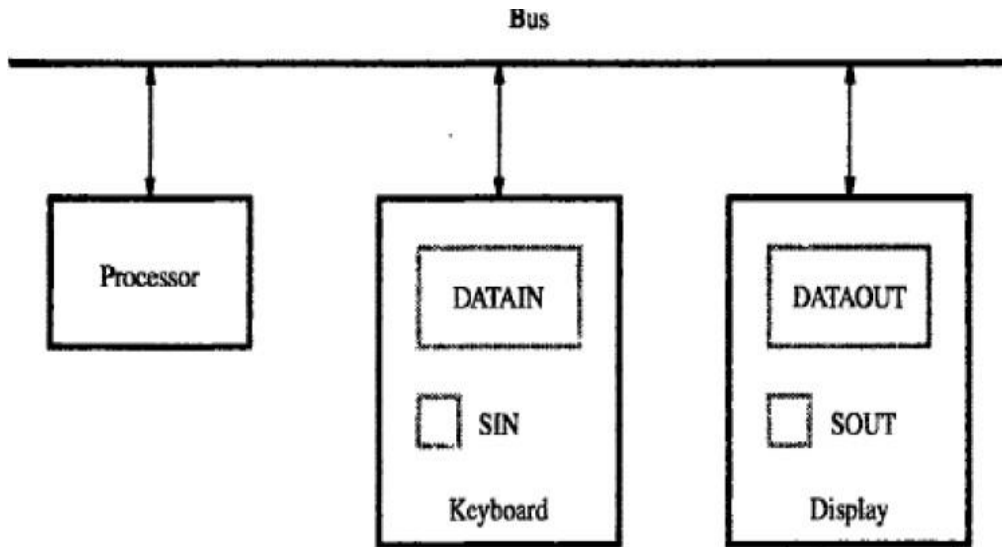


Figure 2.19 Bus connection for processor, keyboard, and display.

Read Operation:

- A program monitors **SIN flag**, and when **SIN is set to 1**, the processor reads the contents of **DATAIN** buffer associated with the keyboard. When the character is transferred to the processor, **SIN** is automatically cleared to 0. If a second character is entered at the keyboard, **SIN** is again set to 1 and the process repeats.
- Let us assume that bit *b3* in **INSTATUS** register corresponds to **SIN flag**. The read operation just described may now be implemented by the machine instruction sequence

Device Interface

Buffer registers: **DATAIN** and **DATAOUT** (8 bits) Status flags: **SIN** and **SOUT**

READWAIT Testbit #3,INSTATUS

Branch=0 READWAIT

MoveByte DATAIN,R1

MoveByte – operation code signifies that the operand size is byte

Programmed I/O

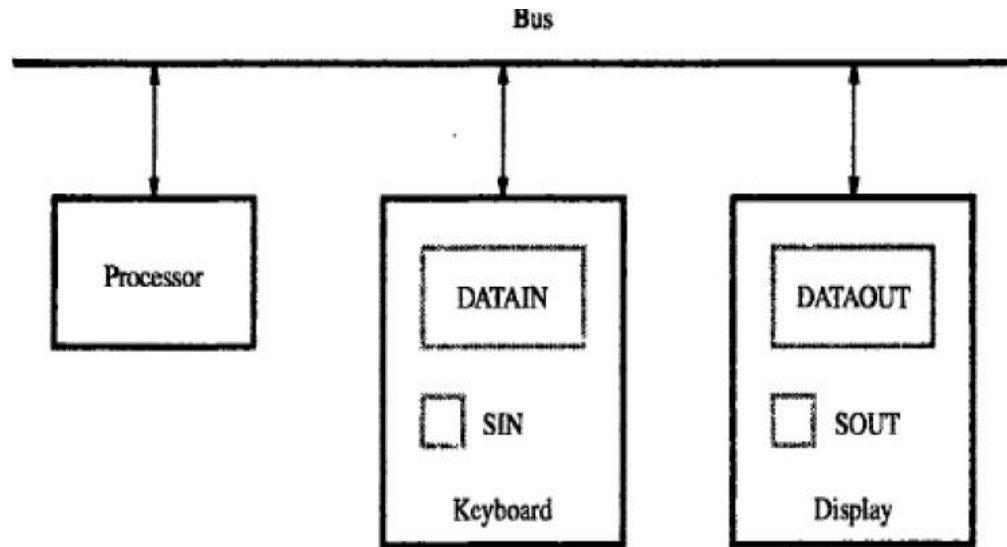


Figure 2.19 Bus connection for processor, keyboard, and display.

Device Interface

Buffer registers: DATAIN and DATAOUT (8 bits)
Status flags: SIN and SOUT

Write Operation:

- A program monitors **SOUT flag**, and when SOUT is set to 1, the processor transfers a character code to **DATAOUT** buffer associated with the display. The transfer of a character to DATAOUT clears SOUT to 0; when the display device is ready to receive a second character, SOUT is again set to 1.
- Let us assume that bit *b3* in OUTSTATUS register corresponds to SOUT flag. The write operation just described may now be implemented by the machine instruction sequence

```
WRITEWAIT Testbit #3,OUTSTATUS  
Branch=0 WRITEWAIT  
MoveByte R1,DATAOUT
```

BASIC INPUT/OUTPUT OPERATIONS

READWAIT Testbit #3,INSTATUS
Branch=0 READWAIT
MoveByte DATAIN,R1

WRITEWAIT Testbit #3,OUTSTATUS
Branch=0 WRITEWAIT
MoveByte R1,DATAOUT

The Testbit instruction tests the state of one bit in the destination location, where the bit position to be tested is indicated by the first operand. If the bit tested is equal to 0, then the condition of the branch instruction is true, and a branch is made to the beginning of the wait loop. When the device is ready, that is, when the bit tested becomes equal to 1, the data are read from the input buffer or written into the output buffer.

Assembly language program to read a line of characters and display it:

	Move	#LOC,R0	Initialize pointer register R0 to point to the address of the first location in memory where the characters are to be stored.
READ	TestBit	#3,INSTATUS	Wait for a character to be entered in the keyboard buffer DATAIN.
	Branch=0	READ	
	MoveByte	DATAIN,(R0)	Transfer the character from DATAIN into the memory (this clears SIN to 0).
ECHO	TestBit	#3,OUTSTATUS	Wait for the display to become ready.
	Branch=0	ECHO	
	MoveByte	(R0),DATAOUT	Move the character just read to the display buffer register (this clears SOUT to 0).
	Compare	#CR,(R0)+	Check if the character just read is CR (carriage return). If it is not CR, then
	Branch≠0	READ	branch back and read another character. Also, increment the pointer to store the next character.

As the characters are read in, one by one, they are stored in a data area in the memory and then echoed back out to the display. The program finishes when carriage return, CR, is read.

The address of the first byte location of the memory where the line is to be stored is LOC.

Figure 2.20 A program that reads a line of characters and displays it.

STACKS

- A *stack* is a list of data elements, usually words or bytes, with the accessing restriction **that elements can be added or removed at one end of the list only**. This end is called the top of the stack, and the other end is called the bottom.
- In order to organize the control and information linkage between the main program and the subroutine, a data structure called a stack is used.
- *Last-in-first-out* (LIFO) - the last data item placed on the stack is the first one removed
- The terms *push* and *pop* are used to describe placing a new item on the stack and removing the top item from the stack, respectively.

STACKS

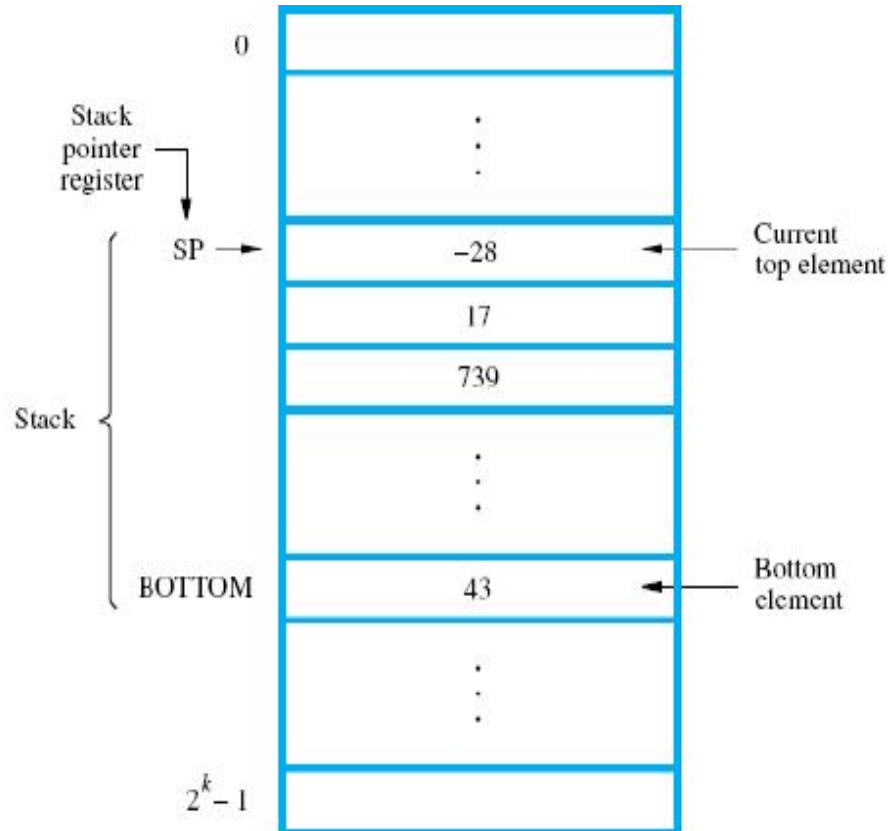


Figure 2.21 A stack of words in the memory.

- Data stored in the memory of a computer can be organized as a stack, with successive elements occupying successive memory locations.
- Assume that the first element is placed in location BOTTOM, and **when new elements are pushed onto the stack, they are placed in successively lower address locations.**
- It contains numerical values, with 43 at the bottom and -28 at the top.
- A processor register is used to keep track of the address of the element of the stack that is **at the top at any given time.** This register is called the **stack pointer (SP)**. It could be one of the general-purpose registers or a register dedicated to this function.

STACKS

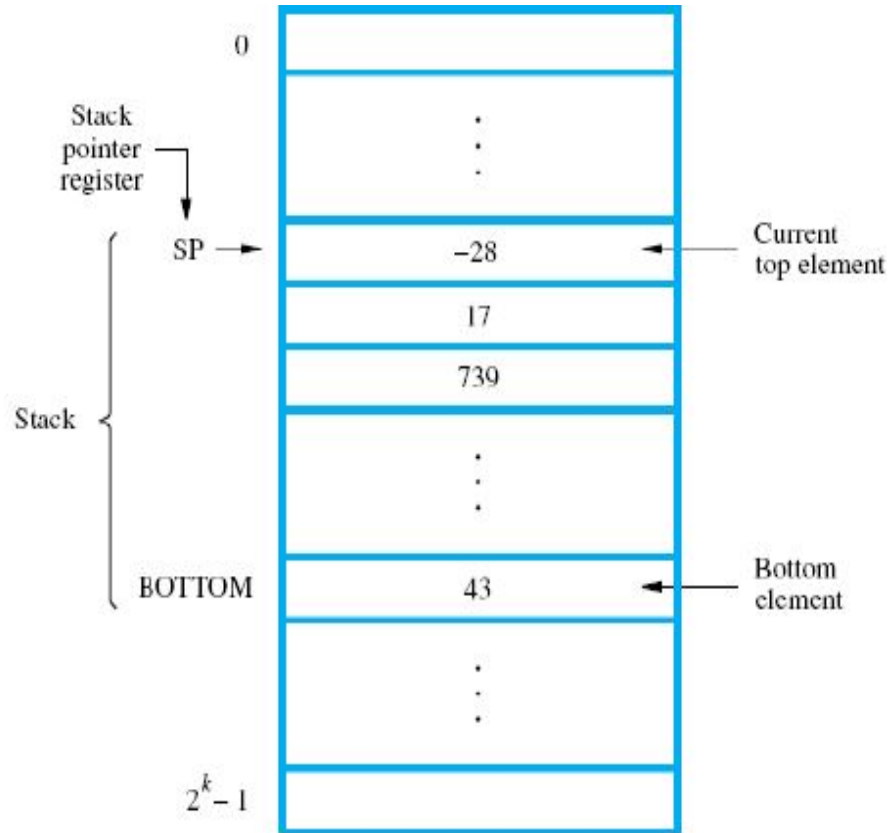


Figure 2.21 A stack of words in the memory.

- Assume, a Byte-addressable memory with a 32-bit word length

push operation

Subtract #4,SP

Move NEWITEM,(SP)

- These two instructions move the word from location NEWITEM onto the top of the stack, decrementing the stack pointer by 4 before the move.

pop operation

Move (SP),ITEM

Add #4,SP

- These two instructions move the top value from the stack into location ITEM and then increment the stack pointer by 4 so that it points to the new top element.

STACKS

- If the processor has the Autoincrement and Autodecrement addressing modes, then the push operation can be performed by the single instruction

Push Operation: Move NEWITEM,−(SP)

Pop Operation: Move (SP)+,ITEM

STACKS

- When a stack is used in a program, it is usually allocated a fixed amount of space in the memory.
- **avoid pushing** an item onto the stack when the stack has reached its maximum size.
- **avoid attempting to pop** an item off an empty stack, which could result from a programming error.
- Suppose that a stack runs from location 2000 (BOTTOM) down no further than location 1500. The stack pointer is loaded initially with the address value 2004. Recall that SP is decremented by 4 before new data are stored on the stack. Hence, an initial value of 2004 means that the first item pushed onto the stack will be at location 2000.

STACKS

- To prevent either pushing an item on a full stack or popping an item off an empty stack, the single-instruction push pop operations can be replaced by the instruction sequences shown in Fig.

SAFEPOP	Compare Branch>0	#2000,SP EMPTYERROR	Check to see if the stack pointer contains an address value greater than 2000. If it does, the stack is empty. Branch to the routine EMPTYERROR for appropriate action.
	Move	(SP)+,ITEM	Otherwise, pop the top of the stack into memory location ITEM.

(a) Routine for a safe pop operation

SAFEPU SH	Compare Branch≤0	#1500,SP FULLERROR	Check to see if the stack pointer contains an address value equal to or less than 1500. If it does, the stack is full. Branch to the routine FULLERROR for appropriate action.
	Move	NEWITEM,−(SP)	Otherwise, push the element in memory location NEWITEM onto the stack.

(b) Routine for a safe push operation

Question

- A processor register is used to keep track of the address of the element of the stack that is at the top at any given time. This register is called the
- Consider the stack in a byte addressable memory with a 32-bit word length and stack runs from location 4000 down no further than location 2500. Write routines for safe push and safe pop operations.

QUEUES

- Data are stored in and retrieved from a queue on a first-in–first-out (FIFO) basis.
- If we assume that the queue grows in the direction of increasing addresses in the memory, new data are added at the back (high-address end) and retrieved from the front (low-address end) of the queue.



QUEUES

There are two important differences between how a stack and a queue are implemented.

STACK

- One end of the stack is fixed (the bottom), while the other end rises and falls as data are pushed and popped.
- A single pointer is needed to point to the top of the stack at any given time.

QUEUE

On the other hand, both ends of a queue move to higher addresses as data are added at the back and removed from the front. So two pointers are needed to keep track of the two ends of the queue.

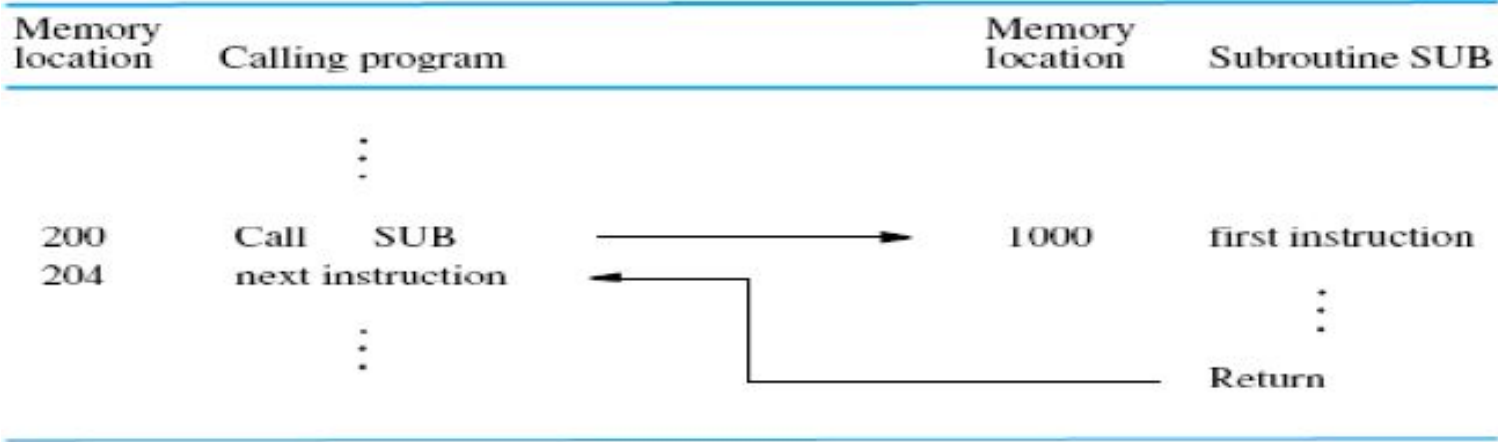
SUBROUTINES

- In a given program, it is often necessary to perform a particular subtask many times on different data values. Such a subtask is usually called a ***subroutine***.

EX: sine function or sort a list of values, sqrt function

- When a program branches to a subroutine we say that it is calling the subroutine. The instruction that performs this branch operation is named a **Call instruction**.
- After a subroutine has been executed, the calling program must resume execution, continuing immediately after the instruction that called the subroutine. The subroutine is said to return to the program that called it by executing a **Return instruction**.

SUBROUTINES



Store the contents of the PC in the **link register**

Branch to the target address specified by the instruction



Branch to the address contained in the link register

Figure 2.24 Subroutine linkage using a link register.

SUBROUTINES

- The way in which a computer makes it possible to call and return from subroutines is referred to as its *subroutine linkage* method.
- The simplest subroutine linkage method is to save the return address in a specific location, which may be a register dedicated to this function. Such a register is called the *link register*. When the subroutine completes its task, the Return instruction returns to the calling program by branching indirectly through the link register.

SUBROUTINES

The Call instruction is just a special branch instruction that performs the following operations:

- Store the contents of the PC in the link register
- Branch to the target address specified by the instruction

The **Return instruction** is a special branch instruction that performs the operation:

- Branch to the address contained in the link register

SUBROUTINE NESTING AND THE PROCESSOR STACK

- A common programming practice, called **subroutine nesting**, is to have one subroutine call another. In this case, the return address of the second call is also stored in the link register, destroying its previous contents. Hence, it is essential to save the contents of the link register in some other location before calling another subroutine. Otherwise, the return address of the first subroutine will be lost.
- Subroutine nesting can be carried out to any depth. Eventually, the last subroutine called completes its computations and returns to the subroutine that called it. The return address needed for this first return is the last one generated in the nested call sequence. That is, return addresses are generated and **used in a last-in-first-out order. This suggests that the return addresses associated with subroutine calls should be pushed onto a stack.**
- .

SUBROUTINE NESTING AND THE PROCESSOR STACK

- A particular register called stack pointer SP is used in this operation. The stack pointer points to a stack called processor stack.
- The call instruction **pushes** the contents of the PC onto the **processor stack** and loads the subroutine address into the PC.
- The return instruction **pops** the return address from the processor stack into the **PC**

PARAMETER PASSING

- When calling a subroutine, a program must provide to the subroutine **the parameters**, that is, the **operands or their addresses**, to be used in the computation
- exchange of information between a calling program and a subroutine is referred to as *parameter passing*.

Parameter passing may be accomplished in several ways. The parameters may be placed

- in registers
- in memory locations
- on the processor stack

Parameter Passing by Value and by Reference

The two parameters, NUM1 and n , passed to the subroutines in Figures 2.25 and 2.26.

passing by reference- calling program passes the address of the first number in the list

passing by value- the actual number of entries, n ,

Passing parameters through processor registers

- Passing parameters through processor registers is straightforward and efficient.

Calling program			
	Move	N,R1	R1 serves as a counter.
	Move	#NUM1,R2	R2 points to the list.
	Call	LISTADD	Call subroutine.
	Move	R0,SUM	Save result.
	:		
Subroutine			
LISTADD	Clear	R0	Initialize sum to 0.
LOOP	Add	(R2)+,R0	Add entry from list.
	Decrement	R1	
	Branch>0	LOOP	
	Return		Return to calling program.

The size of the list, n , contained in memory location N, and the address, NUM1, of the first number, are passed through registers R1 and R2.

The subroutine computes the sum and places it in R0. After the return operation is performed by the subroutine, the sum is stored in memory location SUM by the calling program.

Figure 2.25 Program of Figure 2.16 written as a subroutine; parameters passed through registers.

Passing parameters through processor registers

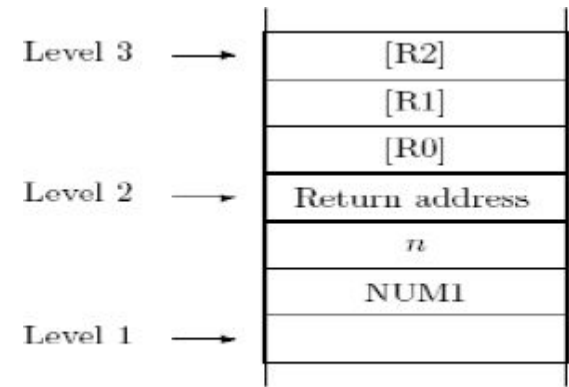
- If many parameters are involved, there may not be enough general-purpose registers available for passing them to the subroutine.
- **Using a stack**, on the other hand, is **highly flexible**; a stack can handle a large number of parameters.

*to add a list
of numbers*

Assume top of stack is at level 1 below.

	Move	#NUM1, -(SP)	Push parameters onto stack
	Move	N, -(SP)	
	Call	LISTADD	Call subroutine (top of stack at level 2).
	Move	4(SP), SUM	Save result.
	Add	#8, SP	Restore top of stack (top of stack at level 1).
	:	:	
LISTADD	MoveMultiple	R0-R2, -(SP)	Save registers (top of stack at level 3).
	Move	16(SP), R1	Initialize counter to n .
	Move	20(SP), R2	Initialize pointer to the list
	Clear	R0	Initialize sum to 0.
LOOP	Add	(R2)+, R0	Add entry from list.
	Decrement	R1	
	Branch > 0	LOOP	
	Move	R0, 20(SP)	Put result on the stack.
	MoveMultiple	(SP)+, R0-R2	Restore registers.
	Return		Return to calling program.

(a) Calling program and subroutine



(b) Top of stack at various times

The subroutine uses three registers. Since these registers may contain valid data that belong to the calling program, their contents should be saved by pushing them onto the stack

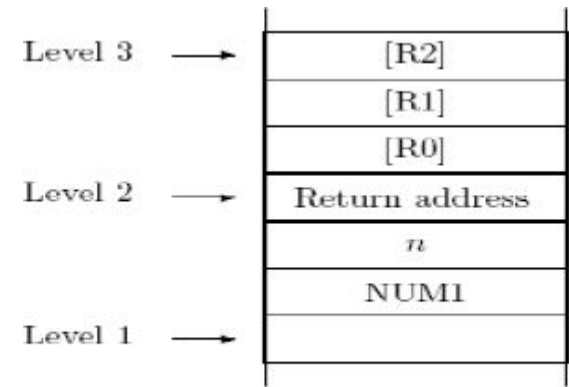
*to add a list
of numbers*

Assume top of stack is at level 1 below.

	Move	#NUM1, -(SP)	Push parameters onto stack
	Move	N, -(SP)	
	Call	LISTADD	Call subroutine (top of stack at level 2).
	Move	4(SP), SUM	Save result.
	Add	#8, SP	Restore top of stack (top of stack at level 1).
	:		
LISTADD	MoveMultiple	R0-R2, -(SP)	Save registers (top of stack at level 3).
	Move	16(SP), R1	Initialize counter to n .
	Move	20(SP), R2	Initialize pointer to the list
	Clear	R0	Initialize sum to 0.
LOOP	Add	(R2)+, R0	Add entry from list.
	Decrement	R1	
	Branch>0	LOOP	
	Move	R0, 20(SP)	Put result on the stack.
	MoveMultiple	(SP)+, R0-R2	Restore registers.
	Return		Return to calling program.

(a) Calling program and subroutine

*The parameters passed to this
subroutine are the address of
the first number in the list and
the number of entries*



(b) Top of stack at various times